



Sistemi Distribuiti

Laurea in Informatica

Marco Savi

marco.savi@unimib.it



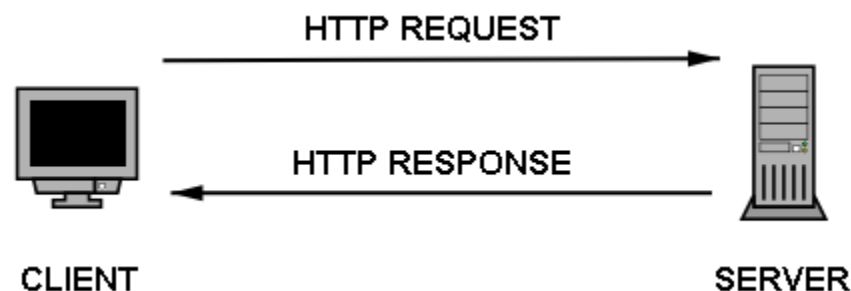
HTTP e message-oriented communication



Breve introduzione al Web

L'architettura del Web

- Il Web è uno dei sistemi distribuiti più complessi
 - Spesso considerato il sistema distribuito per antonomasia
- Segue un modello **client-server**
- Supporta l'interazione tra client e server via HyperText Transfer Protocol (**HTTP**)
 - Protocollo fondamentale del Web



- Il client è tipicamente un **browser**, chiamato anche **user agent**
- Il server è tipicamente un **Web server**, chiamato anche **HTTP server**

Il browser

- Applicazione per il Web lato del client
 - Consente la navigazione nel Web da parte di un utente
- Funzione primaria: interpretare il codice con cui sono espresse le informazioni (**pagine Web**) e visualizzarlo (operazione di **rendering**) in forma di **ipertesto**
- I browser moderni hanno funzioni avanzate per
 - Trattare altri tipi di dati oltre al testo: es. multimedialità, XML, JSON, ... (vedrete più avanti)
 - Ospitare ed eseguire applicazioni: es. JavaScript (vedrete più avanti)
- Il rendering dipende dai dispositivi utilizzati, anche “non visuali”
 - Esempio: supporto a utenti non vedenti (es. sintesi vocale)



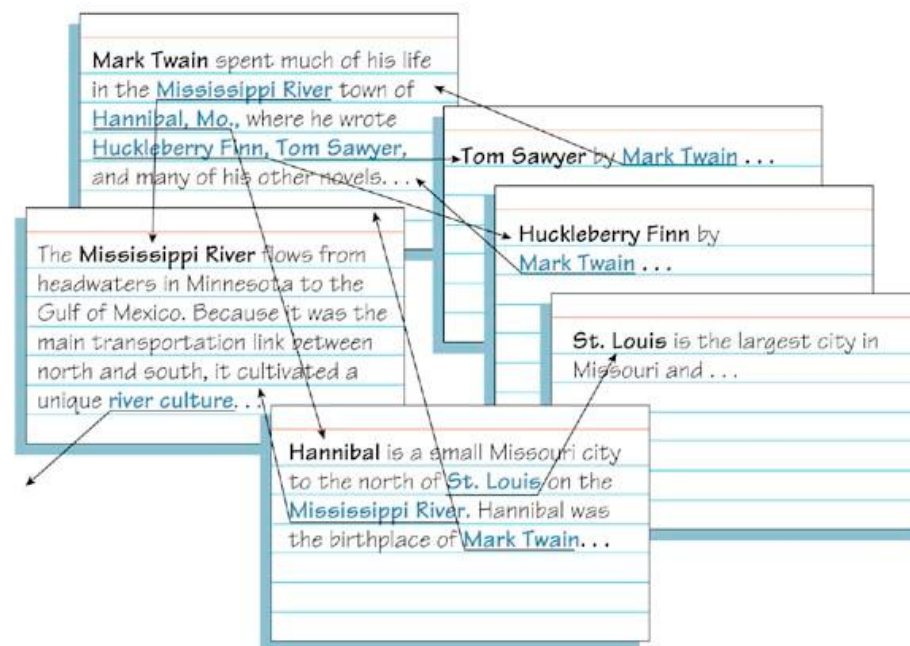
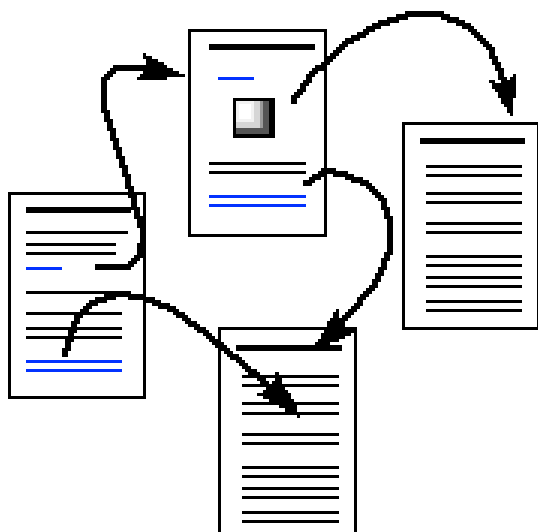
Pagina Web

- E' composta da vari **risorse** (o **oggetti**) delle quali bisogna fare il rendering
 - Testo, immagini, musica, ...
- La **struttura** e i **contenuti** della pagina Web (testo più risorse referenziate) sono definiti in un **file HTML**
 - **HTML**: HyperText Markup Language, è il linguaggio con cui si scrivono gli **ipertesti**
 - Ogni risorsa è identificata da un **Uniform Resource Locator** (univoco per risorsa)
- Importante: il file HTML è a sua volta una risorsa!
- Il **Web server** si occupa di gestire le risorse su un elaboratore e di renderle disponibili ai client



Gli ipertesti

- Il modello di organizzazione delle informazioni fondante del Web è l'ipertesto (**hypertext**)
 - Insieme di contenuti (tipicamente testo, ma non per forza) che possono essere acceduti in maniera **non sequenziale** secondo percorsi definiti da dei **collegamenti ipertestuali** (hyperlink, o più semplicemente **link**)
- Gli ipertesti sono rappresentati da una rete con struttura a **grafo orientato**
 - Caratteristica fondamentale: **non linearità**



Uniform Resource Locator (URL) – RFC 3986

- **Identifica e localizza** univocamente una risorsa nel Web
 - E' un tipo specifico di Uniform Resource Identifier (URI)
- Ha cinque componenti principali:
 1. Nome del protocollo
 - Viene indicato il **protocollo** da usare per ricevere/inviare i dati relativi alla risorsa
 2. Indirizzo IP dell'host (o host name, da risolvere tramite DNS)
 3. Porta di destinazione (opzionale)
 4. Percorso nell'host (segue una gerarchia a cartelle)
 5. Identificatore della risorsa

protocollo : *//* *indirizzo_IP* / *hostname* [*:porta*] / *cammino* / *risorsa*

1. 2. 3. 4. 5.

Uniform Resource Locator (URL) – Esempi (non funzionanti)

`ftp://www.softwarehouse.com/download/software.exe`

`http://www.biblioteca.it/libri/english/book.epub`

`http://www.someSchool.edu/someDept/pic.gif`

`http://www.someSchool.edu:80/someDept/pic.gif`

Identificare senza localizzare: Uniform Resource Name (URN)

- Identificatore **indipendente dalla posizione** della risorsa (RFC 8141)
 - **L'URL** dice **dove** trovare una risorsa, l'**URN** definisce **cosa è** quella risorsa
- Proprietà fondamentali:
 - **Persistenza**: il nome rimane valido anche se la risorsa viene spostata
 - **Astrazione**: non contiene informazioni sul protocollo di accesso (es. HTTP)

- Struttura di un URN:

`urn:<NID>:<NSS>`

- `urn`: identifica il **tipo** di identificatore (sempre fisso)
- `<NID>` è il **Namespace Identifier** (es. `uuid`, `isbn`, `issn`, `ietf`)
 - Identifica il tipo di risorsa
- `<NSS>` è il **Namespace Specific String**
 - Codice identificativo univoco della risorsa

Identificare senza localizzare: Uniform Resource Name (URN)

- L'URN è molto utilizzato nell'**editoria** e nei **sistemi distribuiti** per assegnare identificativi univoci in maniera non coordinata (`uuid`, universally unique identifier)
- Esempi:
 - `urn:isbn:12345`
 - `urn:uuid:f81d4fae-7dec-11d0-a765-00a0c91e6bf6`
- Un URN può essere utilizzato in un URL per identificare la risorsa
- Esempi:
 - `http://biblioteca.it/libri/urn:isbn:9788804668237`
 - `http://miosito.it/user/[urn:uuid:]f47ac10b-58cc-4372-a567-0e02b2c3d479`
 - E' molto comune venga omissso "urn:uuid" in questo caso

I linguaggi del Web

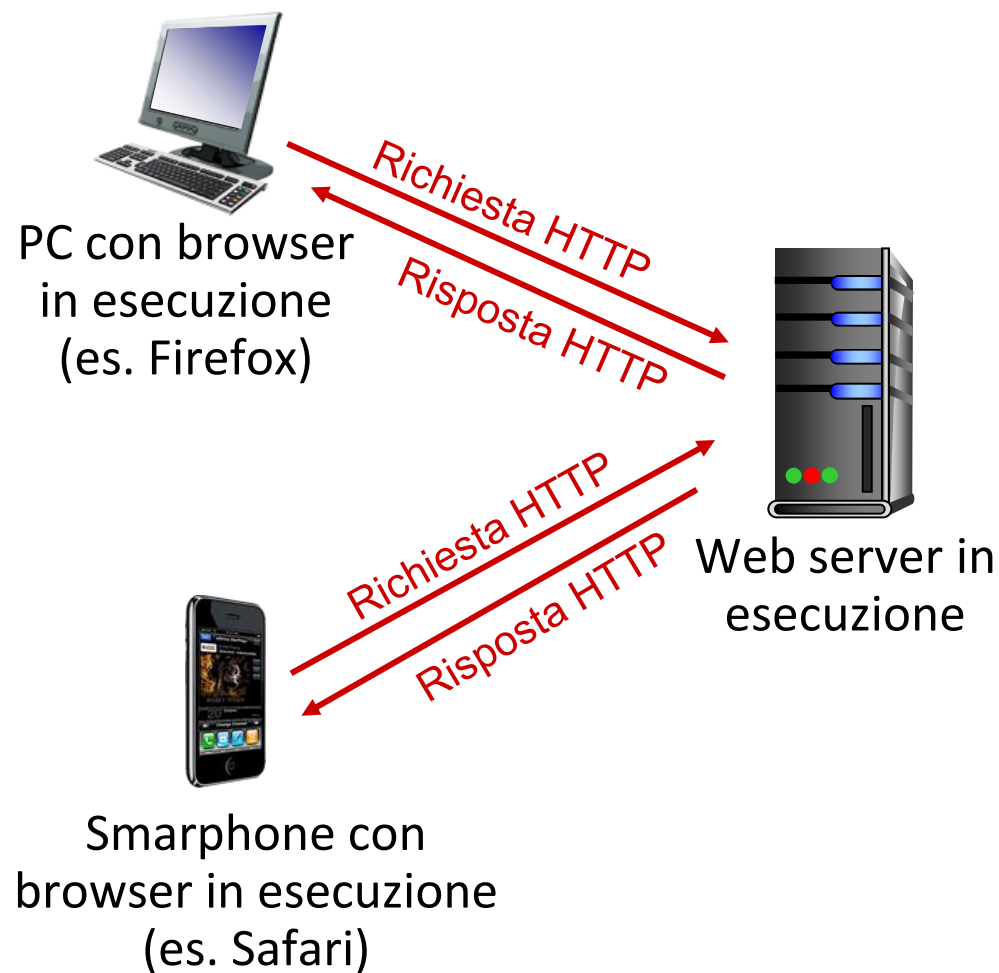
- I **dati testuali** sono espressi in linguaggi standard (vedrete più avanti):
 - HTML (definizione di struttura e i contenuti)
 - Può contenere codice Cascade Style Sheets (CSS) per gestire il rendering (layout, formattazione, aspetto visivo, ...)
 - XML (focalizzato sulla rappresentazione dei dati e loro scambio)
 - JSON (focalizzato sulla rappresentazione dei dati e loro scambio)
- I dati possono essere **non testuali** (immagini, audio, video):
 - Codifica MIME (trasforma i file complessi in una stringa di caratteri testuali per viaggiare sulla rete)
- Le pagine Web possono contenere del codice espresso in **linguaggi di scripting** per arricchire l'interazione (vedrete più avanti):
 - JavaScript, [VBScript, Java/Applet, Adobe Flash], ...



Il protocollo HTTP

Il protocollo HTTP

- **HTTP**: HyperText Transfer Protocol
- Protocollo di livello applicativo per il Web
- Usa il modello **client-server**
 - Client HTTP: browser che richiede, riceve e mostra gli oggetti Web
 - Server HTTP: Web server che invia oggetti in risposta alle richieste
- Nota: oggetti diversi relativi alla stessa pagina Web potrebbero essere reperiti da server Web differenti!
- Varie versioni: **HTTP/1.0** (1996, RFC 1945), **HTTP/1.1** (2014, RFC 7230), **HTTP/2.0** (2015, RFC 7540), **HTTP/3.0** (2022, RFC 9114)
 - HTTP/1.1 la più adottata oggi



Il protocollo HTTP

■ HTTP usa TCP:

- Il client inizia una connessione TCP (crea una socket) verso il server sulla porta 80
- Il server accetta la connessione TCP dal client
- Vengono scambiati messaggi HTTP (di livello applicativo) tra il browser (client HTTP) e il Web server (server HTTP)
- La connessione TCP viene chiusa

■ HTTP è “stateless” (privo di stato):

- Il server non mantiene informazione sulle richieste precedenti del client
- Ogni richiesta deve contenere tutte le informazioni necessarie per la sua esecuzione

I protocolli che mantengono informazione di stato sono complessi (es. TCP)!

HTTP e connessioni TCP: due approcci

■ HTTP non persistente

1. La connessione TCP viene aperta verso un server
2. Al più un oggetto viene inviato dal server al client sfruttando tale connessione
3. La connessione TCP viene chiusa

- Se devono essere scaricati più oggetti sono necessarie più connessioni

- HTTP/1.0 usa connessioni TCP non persistenti

■ HTTP persistente

1. La connessione TCP viene aperta verso un server
2. Più oggetti possono essere inviati dal server al client sfruttando tale connessione
3. La connessione TCP viene chiusa

- HTTP/1.1, HTTP/2.0 e HTTP/3.0 usano connessioni TCP persistenti

Importante: i termini “persistente” e “non persistente” si riferiscono alla connessione

HTTP non persistente: un esempio

L'utente inserisce l'URL: `www.someSchool.edu/someDepartment/home.index`
(contiene testo e referencia 10 immagini jpeg)



tempo



HTTP non persistente: un esempio

L'utente inserisce l'URL: `www.someSchool.edu/someDepartment/home.index`
(contiene testo e referencia 10 immagini jpeg)



1a. Il client HTTP avvia una
connessione TCP verso il server
HTTP a `www.someSchool.edu` sulla
porta 80



1b. Il server HTTP sull'host
`www.someSchool.edu` attende una
connessione TCP sulla porta 80, la
accetta e notifica il client

tempo



HTTP non persistente: un esempio

L'utente inserisce l'URL: `www.someSchool.edu/someDepartment/home.index`
(contiene testo e referencia 10 immagini jpeg)



HTTP non persistente: un esempio

L'utente inserisce l'URL: `www.someSchool.edu/someDepartment/home.index`
(contiene testo e referencia 10 immagini jpeg)



tempo



HTTP non persistente: un esempio

L'utente inserisce l'URL: `www.someSchool.edu/someDepartment/home.index`
(contiene testo e referencia 10 immagini jpeg)



4. Il server chiude la
connessione TCP

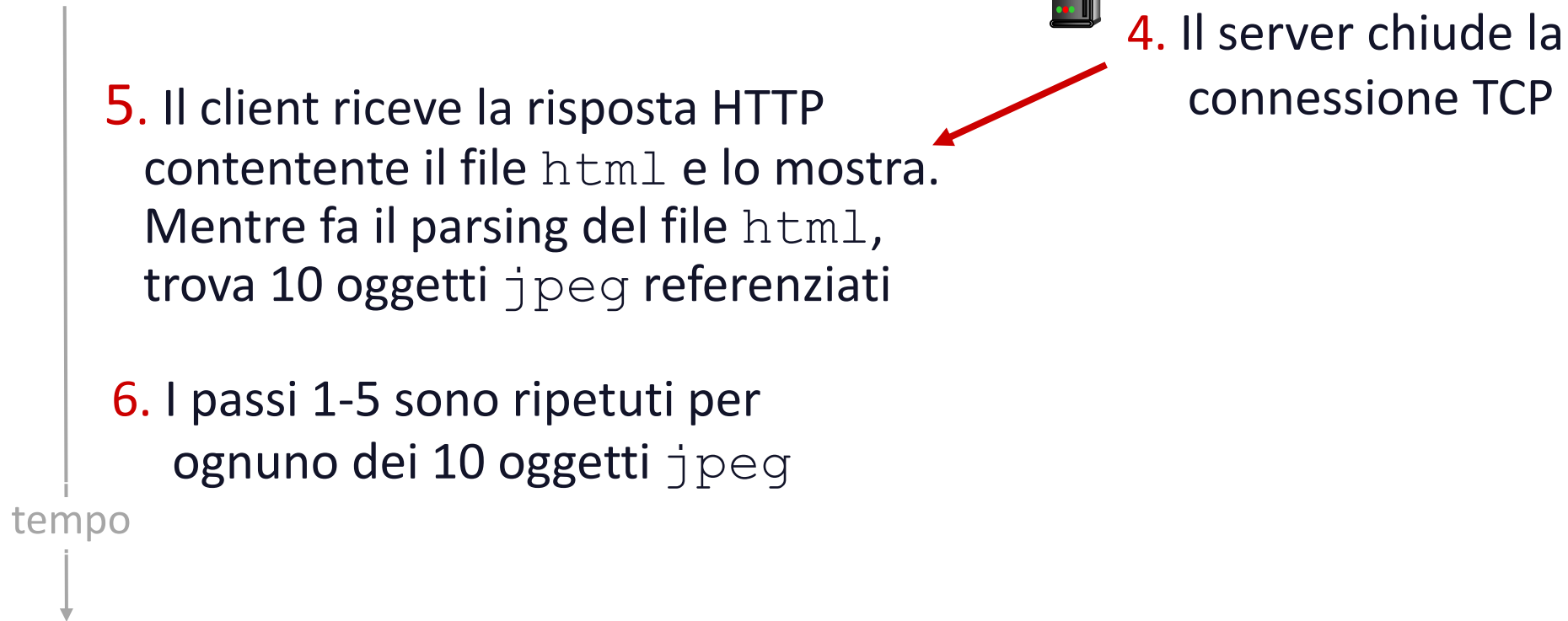
5. Il client riceve la risposta HTTP
contentente il file `html` e lo mostra.
Mentre fa il parsing del file `html`,
trova 10 oggetti `jpeg` referenziati

tempo



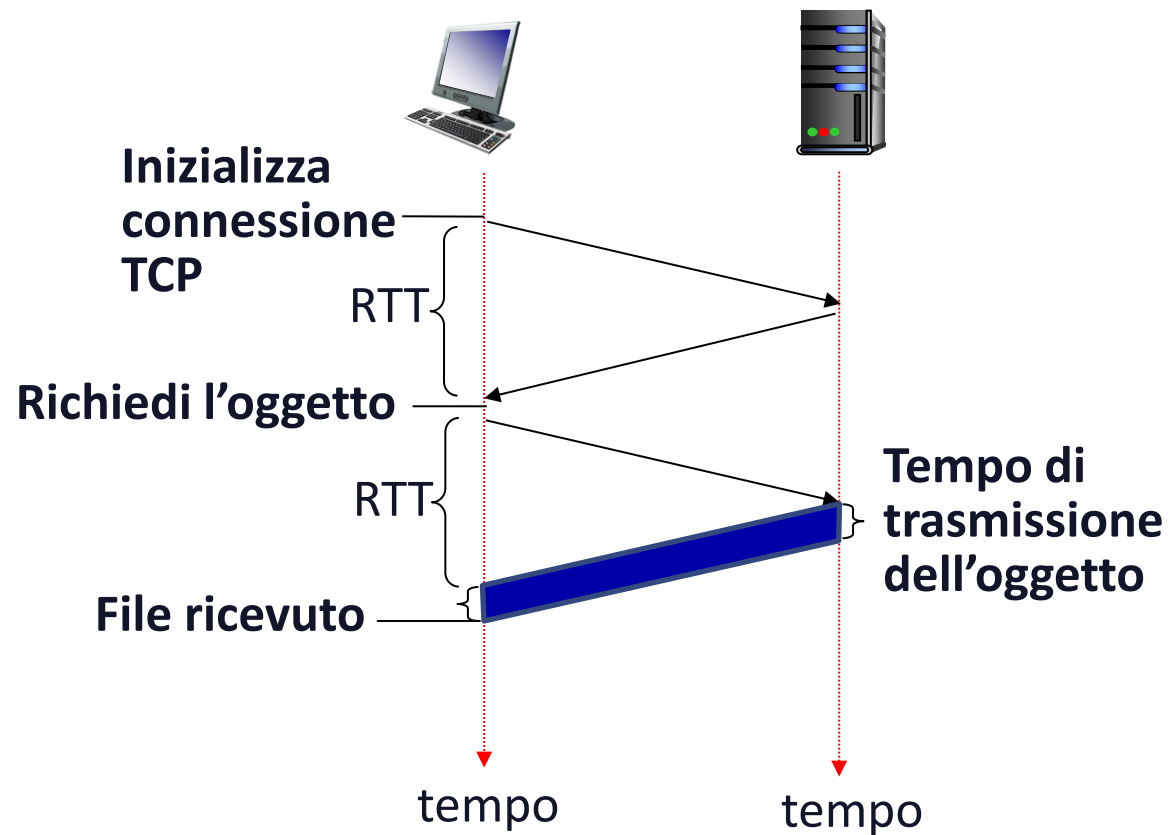
HTTP non persistente: un esempio

L'utente inserisce l'URL: `www.someSchool.edu/someDepartment/home.index`
(contiene testo e referencia 10 immagini jpeg)



HTTP non persistente: tempo di risposta

- Tempo di risposta per ogni oggetto:
 - Un Round Trip Time (RTT) per inizializzare la connessione
 - Un RTT per inviare la richiesta HTTP
 - Tempo di trasmissione dell'oggetto
- $2\text{RTT} + \text{tempo di trasmissione dell'oggetto}$
- Spesso i browser aprono connessioni TCP in parallelo
 - Necessario aprire più socket (sia lato client che lato server)
 - Impatto negativo sul controllo di congestione (uso non equo delle risorse)
- Se $\text{RTT} \gg \text{tempo trasmissione}$, **HTTP persistente** all'incirca dimezza il tempo di risposta per oggetto (se si hanno tanti oggetti)



Formato dei messaggi HTTP – Richieste HTTP

- HTTP è un protocollo orientato al carattere → Formato messaggi: ASCII (testo leggibile)
- Due tipi di messaggi HTTP: *richiesta* (request), *risposta* (response)
 - Hanno la stessa struttura!
- **Messaggio di richiesta HTTP:**

Formato dei messaggi HTTP – Richieste HTTP

- HTTP è un protocollo orientato al carattere → Formato messaggi: ASCII (testo leggibile)
- Due tipi di messaggi HTTP: *richiesta* (request), *risposta* (response)
 - Hanno la stessa struttura!
- **Messaggio di richiesta HTTP:**

riga di richiesta
(metodo, URL,
versione)

→ GET /index.html HTTP/1.1\r\n

carattere di ritorno a capo (cr)
carattere di nuova linea (lf)

Formato dei messaggi HTTP – Richieste HTTP

- HTTP è un protocollo orientato al carattere → Formato messaggi: ASCII (testo leggibile)
- Due tipi di messaggi HTTP: *richiesta* (request), *risposta* (response)
 - Hanno la stessa struttura!

- **Messaggio di richiesta HTTP:**

riga di richiesta

(metodo, URL,
versione)

righe di intestazione

carattere di ritorno a capo (cr)

carattere di nuova linea (lf)

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X
10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Connection: keep-alive\r\n
\r\n
```

la coppia {ritorno a capo,
nuova linea} all'inizio di
una linea indica la fine
dell'intestazione

Formato dei messaggi HTTP – Richieste HTTP

- HTTP è un protocollo orientato al carattere → Formato messaggi: ASCII (testo leggibile)
- Due tipi di messaggi HTTP: *richiesta* (request), *risposta* (response)
 - Hanno la stessa struttura!
- **Messaggio di richiesta HTTP:**

specifica l'host su cui
risiede l'oggetto

```
GET /index.html HTTP/1.1\r\nHost: www-net.cs.umass.edu\r\n
```

specifica il tipo di browser

```
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X  
10.15; rv:80.0) Gecko/20100101 Firefox/80.0 \r\n
```

negoziiazione del contenuto
(formato MIME)

```
Accept: text/html,application/xhtml+xml\r\n
```

```
Accept-Language: en-us,en;q=0.5\r\n
```

```
Accept-Encoding: gzip,deflate\r\n
```

preferenza sulla lingua

```
Connection: keep-alive\r\n
```

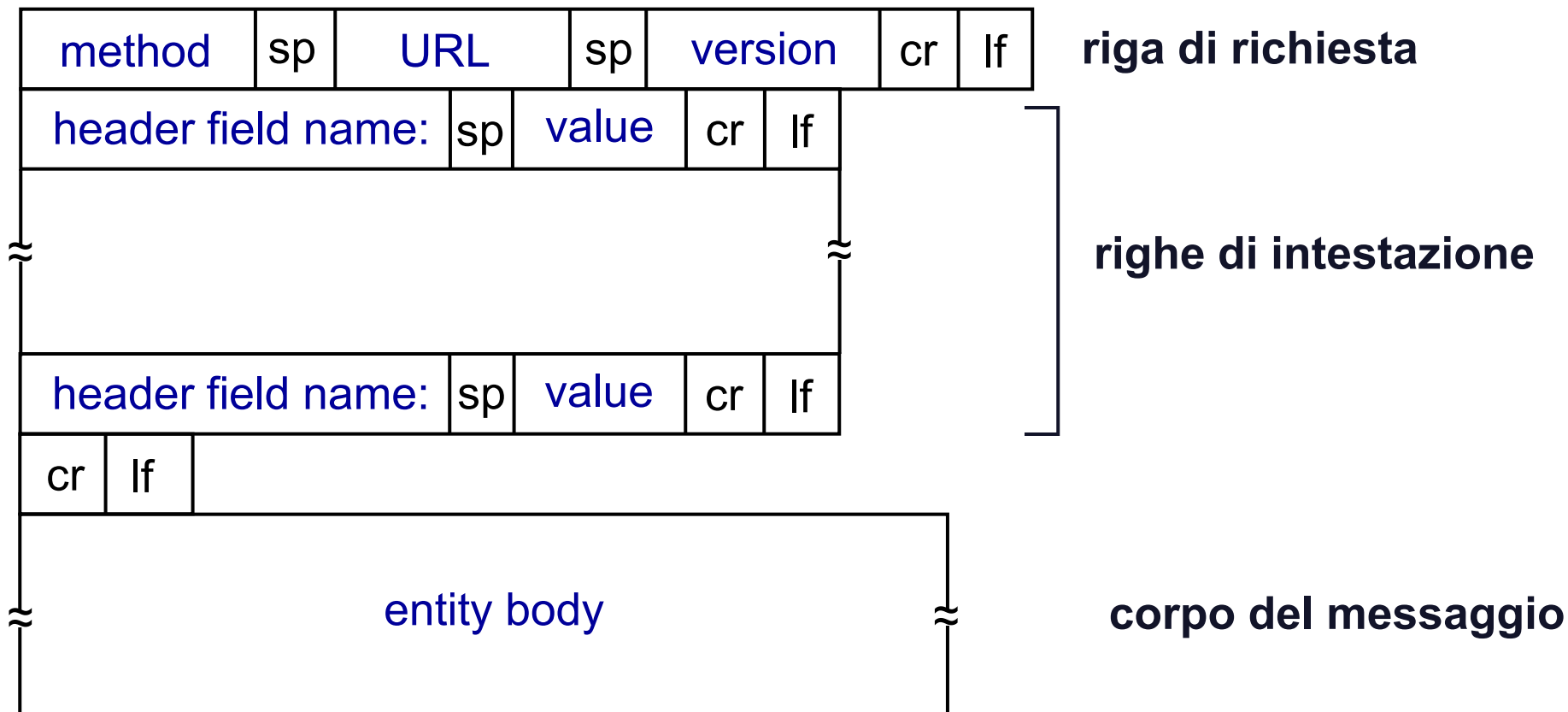
```
\r\n
```

preferenza di compressione

richiesta di connessione
persistente

Richieste HTTP: formato generale

cr: carriage return (ritorno a capo)
lf: line feed (nuova linea)



Richieste HTTP: metodi principali

■ Metodo GET

- Richiede la risorsa specificata dal URL
 - Include eventuali parametri in coda al URL della risorsa
- Uso tipico: ottenere dati in vari formati (HTML, XML, JSON, oppure multimediali)

■ Metodo HEAD

- Simile al metodo GET ma viene restituita solo l'intestazione associata alla risorsa
- Uso tipico: debugging o ottenere informazioni sulla risorsa (es. la sua dimensione)

■ Metodo POST

- Comunica dei dati da elaborare lato server
 - Dati inseriti nel corpo della richiesta
- Uso tipico: trasportare i dati inseriti in un form

■ Metodo PUT

- Carica un nuovo oggetto sul server all'URL specificato (se necessario sostituisce l'oggetto esistente)
 - Oggetto inserito nel corpo della richiesta
- Uso tipico: sovrascrittura di credenziali

GET e POST: passaggio parametri

- La GET include eventuali parametri in coda alla URL della risorsa (detti *query string*)
 - ? è il separatore primario (tutto ciò che è dopo ? sono parametri)
 - & è un connettore, usato per delimitare più parametri che devono essere passati

```
GET resource[?key=value{&key=value}] HTTP1.1
{header lines}
```

- Esempio: dammi tutti gli ordini di marzo per il prodotto con codice Q2345

```
GET myCompany/orders?item="Q2345"&date="2022/03" HTTP1.1
```

- La POST prevede che i dati vengano messi dopo l'intestazione come documento autonomo (detto *body*)

```
POST resource HTTP1.1
{header lines}
body
```

- Esempio: aggiorna i codici di un prodotto in tutti gli ordini di aprile

```
POST myCompany/orders HTTP1.1
Content-Length: 59
Content-Type: application/x-www-form-urlencoded
update=true&oldItem="Q2345"&newItem="Q68254"&date="2022/04"
```

Metodi HTTP

Metodo	Descrizione	Sicuro	Idempotente
OPTIONS	Rappresenta una richiesta di informazioni sulle opzioni di comunicazione relativamente alla risorsa identificata dal URL.	✓	✓
GET	Recupera qualunque informazione relativa alla risorsa identificata dal URL e la risorsa stessa.	✓	✓
HEAD	Identico a GET, tranne per il fatto che il server NON deve restituire un corpo del messaggio nella risposta.	✓	✓
POST	Usato per richiedere che il server accetti la risorsa inclusa nella richiesta come un nuovo subordinato della risorsa identificata dal URL.		
PUT	Richiede che la risorsa fornita venga memorizzata sotto l'URL indicato. Se un file con quello stesso URL esiste già, viene completamente sostituito dal contenuto presente nel corpo della richiesta.		✓
DELETE	Richiede che il server elimini la risorsa identificata dal URL.		✓
TRACE	Usato per invocare un loopback del messaggio di richiesta a livello applicativo da parte del server.	✓	✓

Sicuro: non modifica lo stato del server

Idempotente: l'effetto sul server di una singola richiesta eseguita con successo è **identico** all'effetto di più richieste identiche eseguite consecutivamente

<http://tools.ietf.org/html/rfc2616>

Formato dei messaggi HTTP – Risposte HTTP

- **Messaggio di risposta HTTP:**

riga di stato (versione, codice di stato, descrizione)  **HTTP/1.1 200 OK**

Formato dei messaggi HTTP – Risposte HTTP

■ Messaggio di risposta HTTP:

riga di stato (versione, codice
di stato, descrizione)

righe di intestazione

HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
OpenSSL/1.0.2k-fips PHP/7.4.9
mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n

Formato dei messaggi HTTP – Risposte HTTP

■ Messaggio di risposta HTTP:

riga di stato (versione, codice
di stato, descrizione)

righe di intestazione

dati (es. file HTML richiesto)

```
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2020 00:53:20 GMT
Server: Apache/2.4.6 (CentOS)
       OpenSSL/1.0.2k-fips PHP/7.4.9
       mod_perl/2.0.11 Perl/v5.16.3
Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
ETag: "a5b-52d015789ee9e"
Accept-Ranges: bytes
Content-Length: 2651
Content-Type: text/html; charset=UTF-8
\r\n
data data data data data ...
```

Formato dei messaggi HTTP – Risposte HTTP

■ Messaggio di risposta HTTP:

	HTTP/1.1 200 OK
momento di generazione della risposta	→ Date: Tue, 08 Sep 2020 00:53:20 GMT
dettagli del software lato server	→ Server: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.4.9 mod_perl/2.0.11 Perl/v5.16.3
ultima modifica dell'oggetto	→ Last-Modified: Tue, 01 Mar 2016 18:57:50 GMT
identificatore univoco dell'oggetto	→ ETag: "a5b-52d015789ee9e"
il server supporta il download parziale	→ Accept-Ranges: bytes
dimensione del corpo del messaggio (in byte)	→ Content-Length: 2651
tipo del contenuto (pagina HTML) con codifica caratteri UTF-8	→ Content-Type: text/html; charset=UTF-8 \r\n data data data data data ...

Risposte HTTP: codici di stato

- Cinque diverse classi:
 - 1xx (Informational): richiesta ricevuta, il server sta continuando l'elaborazione
 - 2xx (Success): richiesta ricevuta, compresa, accettata ed elaborata
 - 3xx (Redirection): necessario intraprendere ulteriori azioni per completare la richiesta
 - 4xx (Client Error): richiesta con sintassi errata oppure non comprensibile
 - 5xx (Server Error): il server non è riuscito a soddisfare una richiesta apparentemente valida
- Alcuni esempi comuni:
 - 200 OK: successo. Se richiesto, l'oggetto è contenuto nel messaggio
 - 301 Moved Permanently: l'oggetto richiesto è stato spostato. Il nuovo indirizzo è specificato nel'header (`Location: ...`)
 - 400 Bad Request: richiesta incomprensibile al server
 - 404 Not Found: l'oggetto non è stato trovato sul server
 - 505 HTTP Version Not Supported: il server non supporta la versione HTTP

Tipi MIME (Multipurpose Internet Mail Extension, RFC 2045)

- Sono delle etichette standardizzate che servono a identificare il formato dei dati
 - L'encoding MIME trasforma quindi i dati in una opportuna stringa di caratteri in base al tipo
- Esempi (visti in precedenza):
 - `Accept: text/html,application/xhtml+xml` (richiesta)
 - `Content-Type: text/html` (risposta)
- 5 categorie che possono essere specializzate con sottotipi (carattere / delimitatore tra tipo e sottotipo)

Text

- Esempi di sottotipi: `plain,html`

Image

- Esempi di sottotipi : `jpeg,gif`

Audio

- Esempio di sottotipo: `mpeg,wav,midi`

Video

- Esempio di sottotipo: `mpeg,mov,mp4`

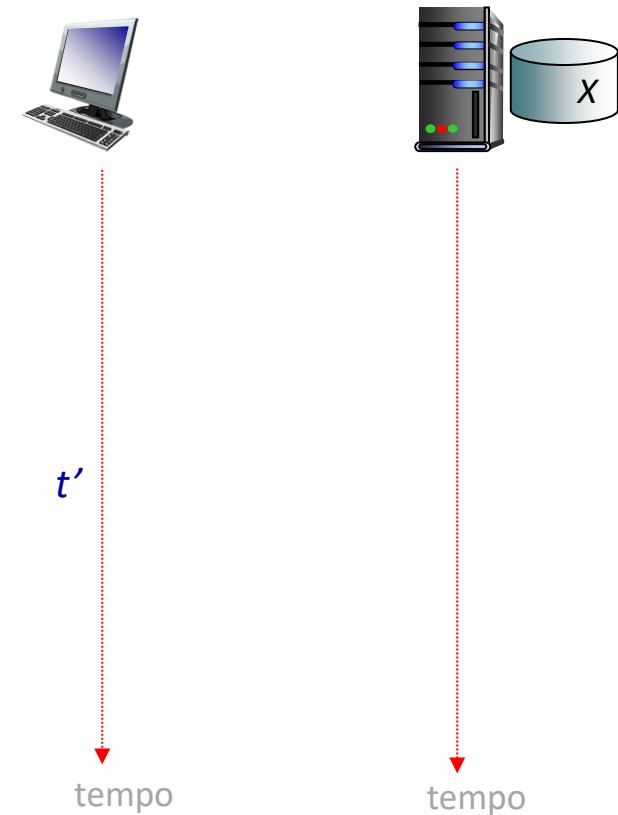
Application: dati che devono essere elaborati da un'applicazione prima di essere renderizzati

- Esempi di sottotipo: `pdf,zip,json`

Mantenere lo stato in HTTP: i cookies

- L'interazione GET/risposta HTTP è **stateless** (senza mantenimento di stato)
 - Tutte le richieste HTTP sono indipendenti l'una dall'altra
- Vantaggi:
 - Non è necessario mettere in atto meccanismi complessi per tener traccia dello stato in uno scambio su più passi (“**transazione**”)
 - Non è necessario che client e server sappiano recuperare da una transazione solo parzialmente completata
- Svantaggi:
 - I messaggi hanno un overhead maggiore (devono includere tutte le informazioni per l'elaborazione)
 - Maggiore complessità lato sviluppo

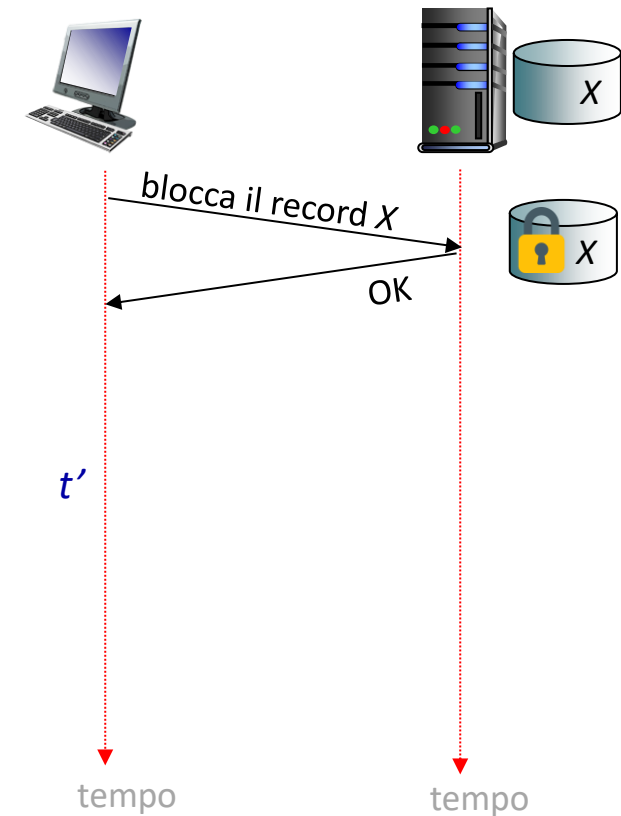
esempio di transazione stateful: il client effettua due modifiche all'oggetto X, oppure nessuna



Mantenere lo stato in HTTP: i cookies

- L'interazione GET/risposta HTTP è **stateless** (senza mantenimento di stato)
 - Tutte le richieste HTTP sono indipendenti l'una dall'altra
- Vantaggi:
 - Non è necessario mettere in atto meccanismi complessi per tener traccia dello stato in uno scambio su più passi (“**transazione**”)
 - Non è necessario che client e server sappiano recuperare da una transazione solo parzialmente completata
- Svantaggi:
 - I messaggi hanno un overhead maggiore (devono includere tutte le informazioni per l'elaborazione)
 - Maggiore complessità lato sviluppo

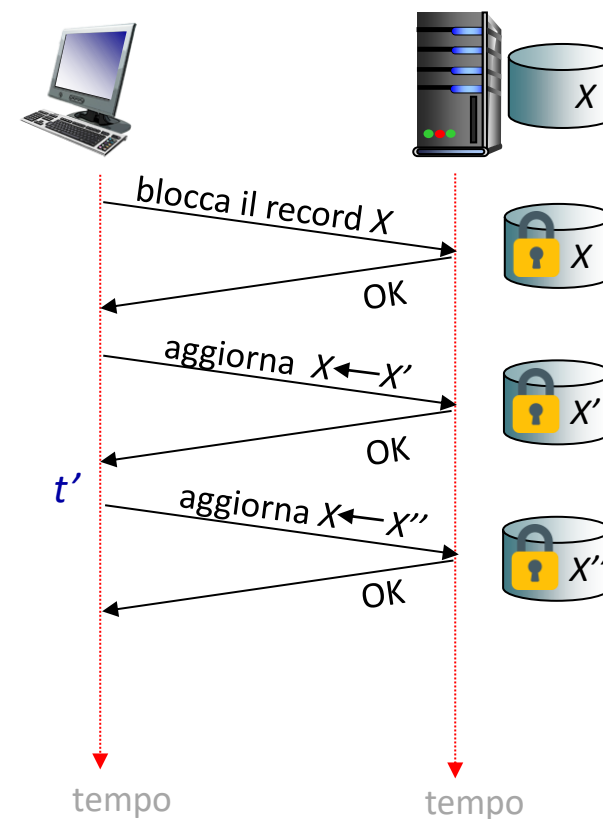
esempio di transazione stateful: il client effettua due modifiche all'oggetto X, oppure nessuna



Mantenere lo stato in HTTP: i cookies

- L'interazione GET/risposta HTTP è **stateless** (senza mantenimento di stato)
 - Tutte le richieste HTTP sono indipendenti l'una dall'altra
- Vantaggi:
 - Non è necessario mettere in atto meccanismi complessi per tener traccia dello stato in uno scambio su più passi ("**transazione**")
 - Non è necessario che client e server sappiano recuperare da una transazione solo parzialmente completata
- Svantaggi:
 - I messaggi hanno un overhead maggiore (devono includere tutte le informazioni per l'elaborazione)
 - Maggiore complessità lato sviluppo

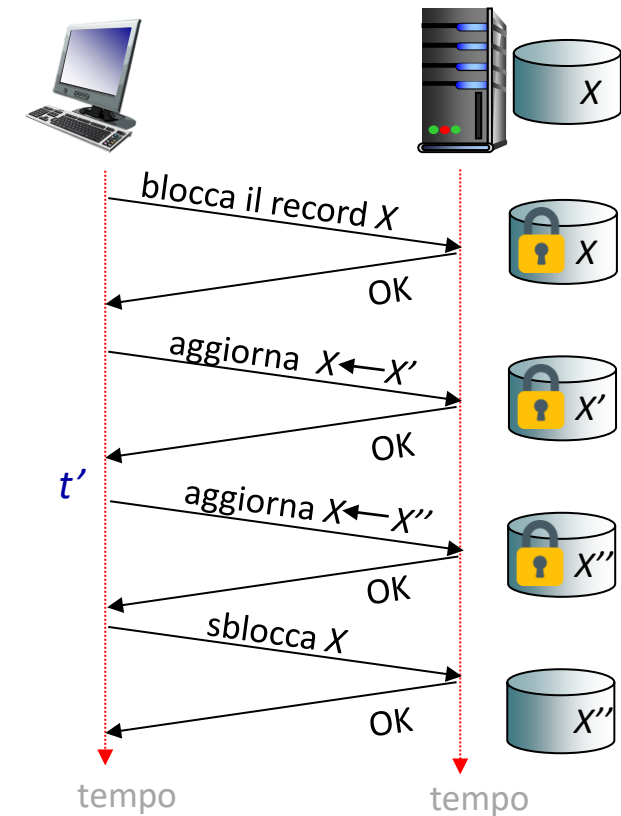
esempio di transazione stateful: il client effettua due modifiche all'oggetto X, oppure nessuna



Mantenere lo stato in HTTP: i cookies

- L'interazione GET/risposta HTTP è **stateless** (senza mantenimento di stato)
 - Tutte le richieste HTTP sono indipendenti l'una dall'altra
- Vantaggi:
 - Non è necessario mettere in atto meccanismi complessi per tener traccia dello stato in uno scambio su più passi ("**transazione**")
 - Non è necessario che client e server sappiano recuperare da una transazione solo parzialmente completata
- Svantaggi:
 - I messaggi hanno un overhead maggiore (devono includere tutte le informazioni per l'elaborazione)
 - Maggiore complessità lato sviluppo

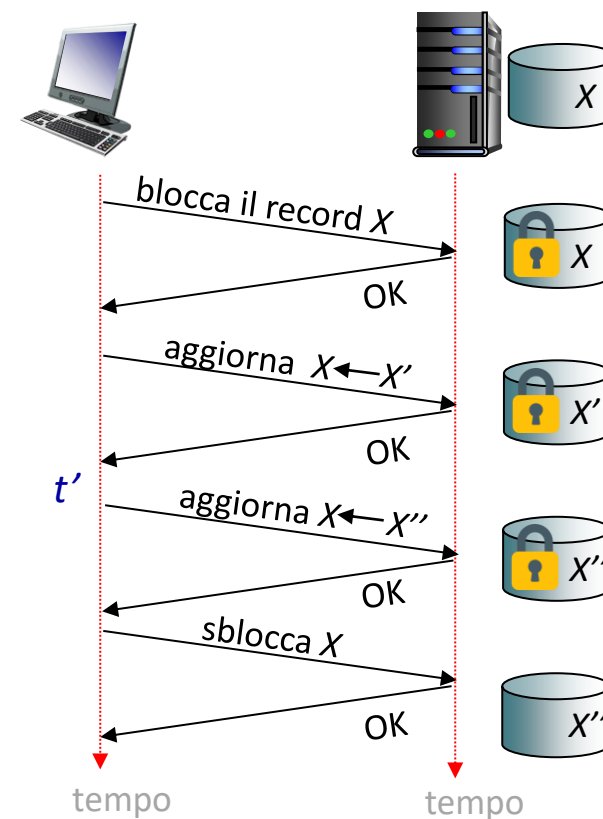
esempio di transazione stateful: il client effettua due modifiche all'oggetto X, oppure nessuna



Mantenere lo stato in HTTP: i cookies

- L'interazione GET/risposta HTTP è **stateless** (senza mantenimento di stato)
 - Tutte le richieste HTTP sono indipendenti l'una dall'altra
- Vantaggi:
 - Non è necessario mettere in atto meccanismi complessi per tener traccia dello stato in uno scambio su più passi ("**transazione**")
 - Non è necessario che client e server sappiano recuperare da una transazione solo parzialmente completata
- Svantaggi:
 - I messaggi hanno un overhead maggiore (devono includere tutte le informazioni per l'elaborazione)
 - Maggiore complessità lato sviluppo

esempio di transazione stateful: il client effettua due modifiche all'oggetto X, oppure nessuna



Cosa succede se la connessione del client cade al tempo t' ?

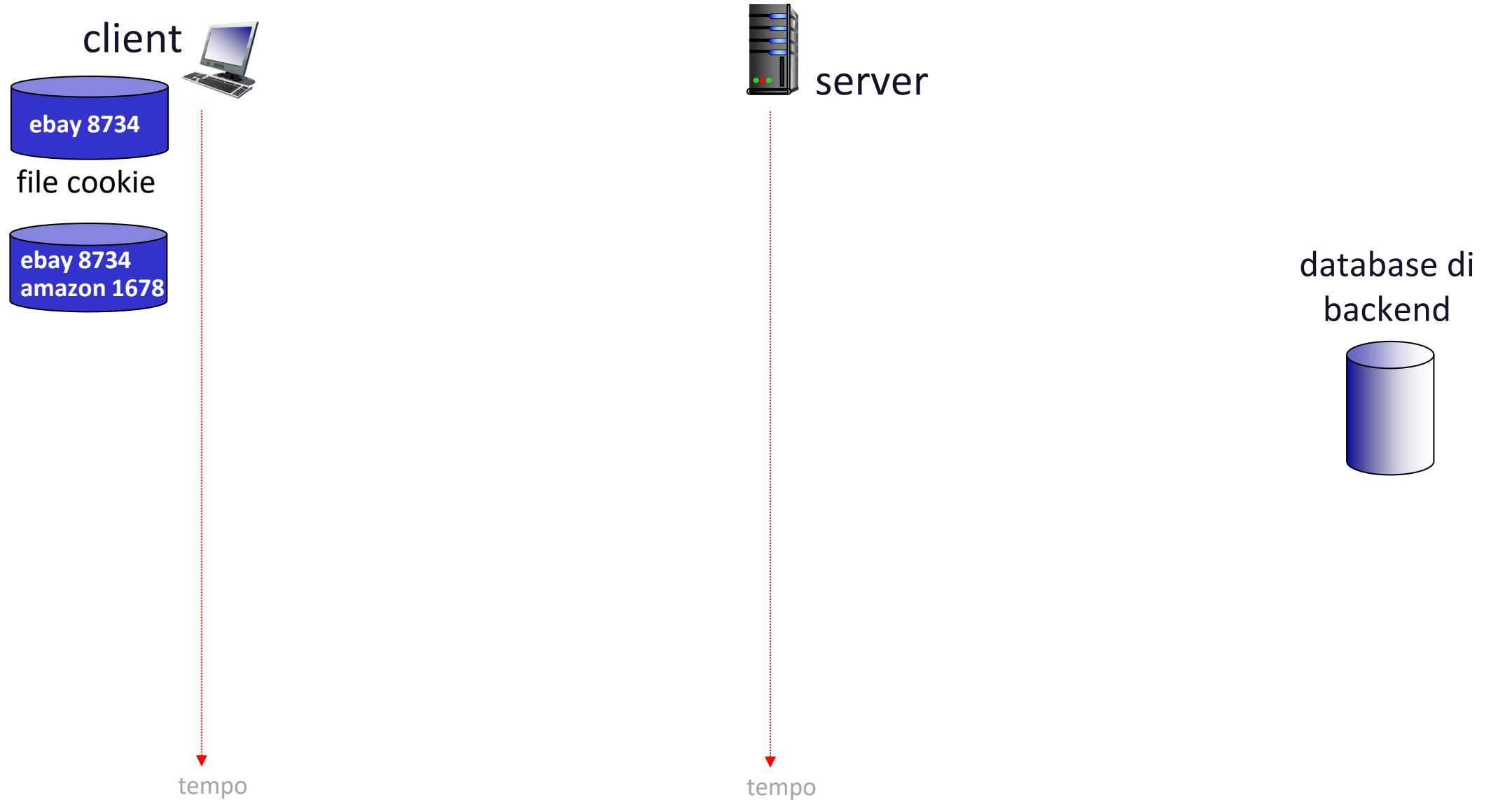
Mantenere lo stato in HTTP: i cookies

- I client e server HTTP usano i **cookie** per il mantenimento dello stato nelle transazioni
- Quattro componenti:
 1. Riga di intestazione specifica nella risposta HTTP
 2. Riga di intestazione specifica nella richiesta HTTP seguente
 3. File relativo al cookie mantenuto dal client e gestito dal browser
 4. Database di backend mantenuto e gestito dal Web server
- Principali utilizzi:
 - Autorizzazione/autenticazione
 - Carrello
 - Raccomandazioni (es. pubblicità mirate)

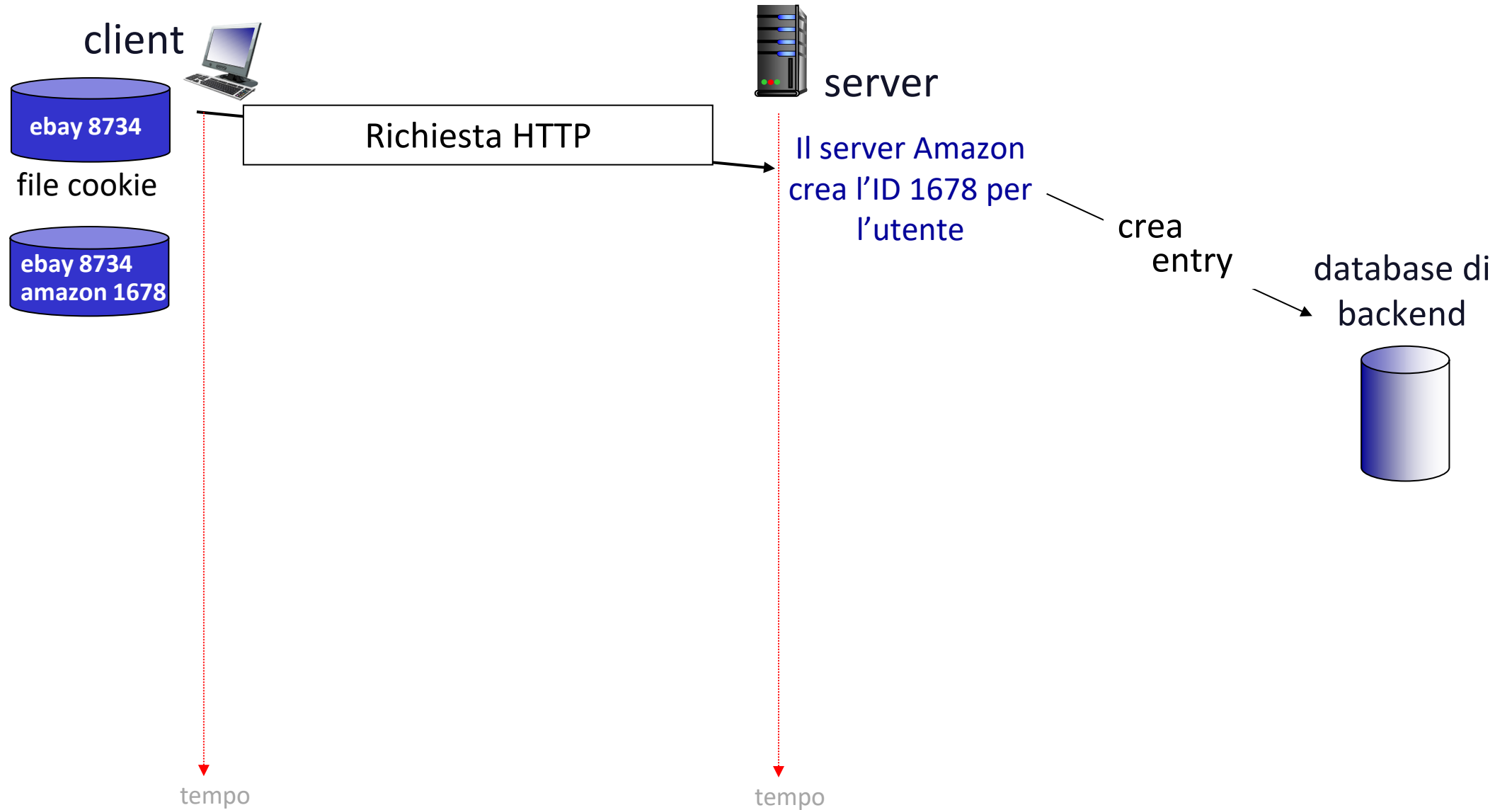
Esempio:

- Susan usa il browser sul suo PC e visita un sito di e-commerce per la prima volta
- Quando la prima richiesta HTTP arriva al Web server, questo crea:
 - Un ID univoco (il cookie)
 - Una entry nel database di backend per ID
- Le richieste HTTP seguenti da parte di Susan includeranno l'ID del cookie
 - Possibile identificare la transazione

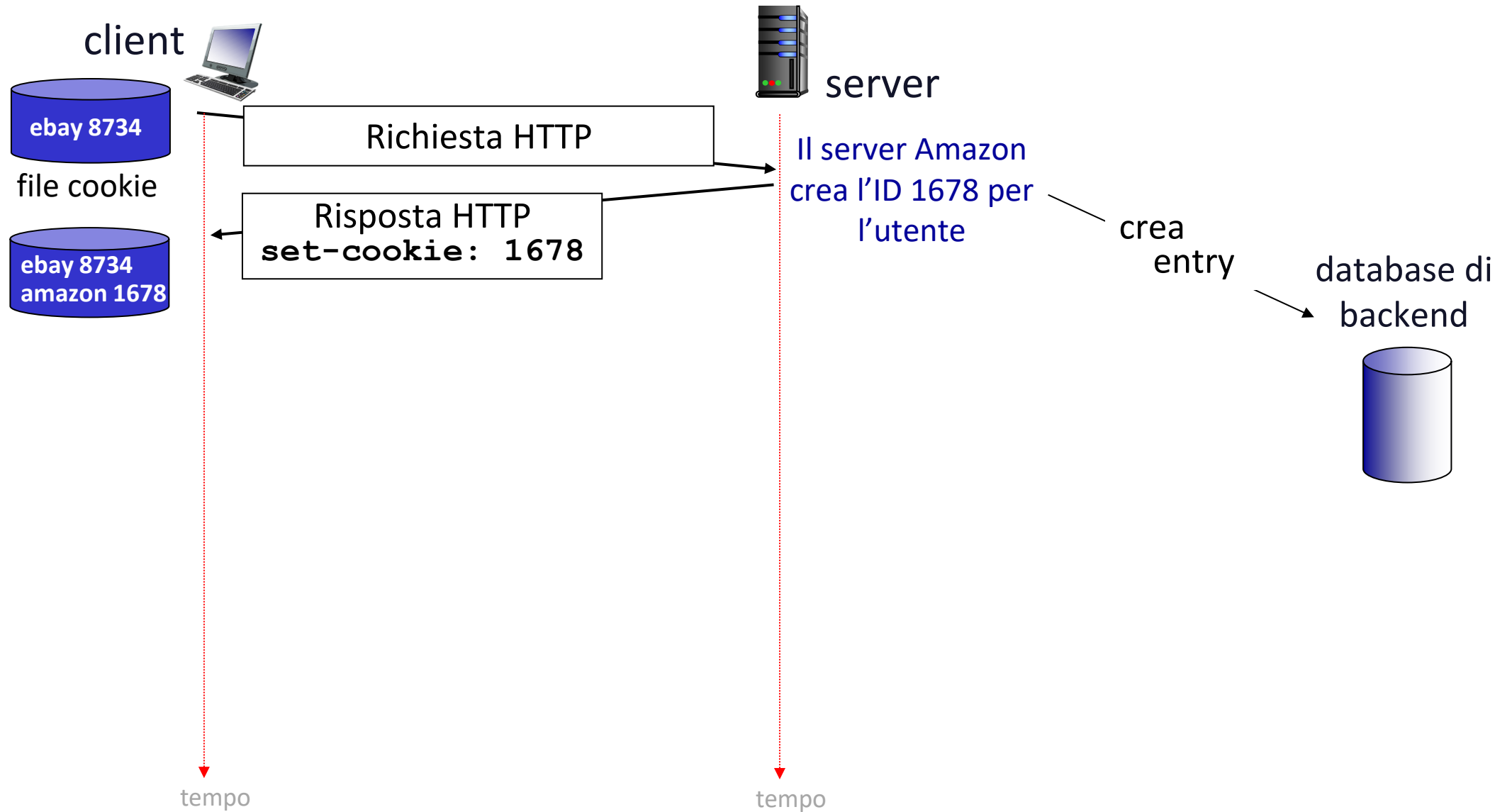
Mantenere lo stato in HTTP: i cookies



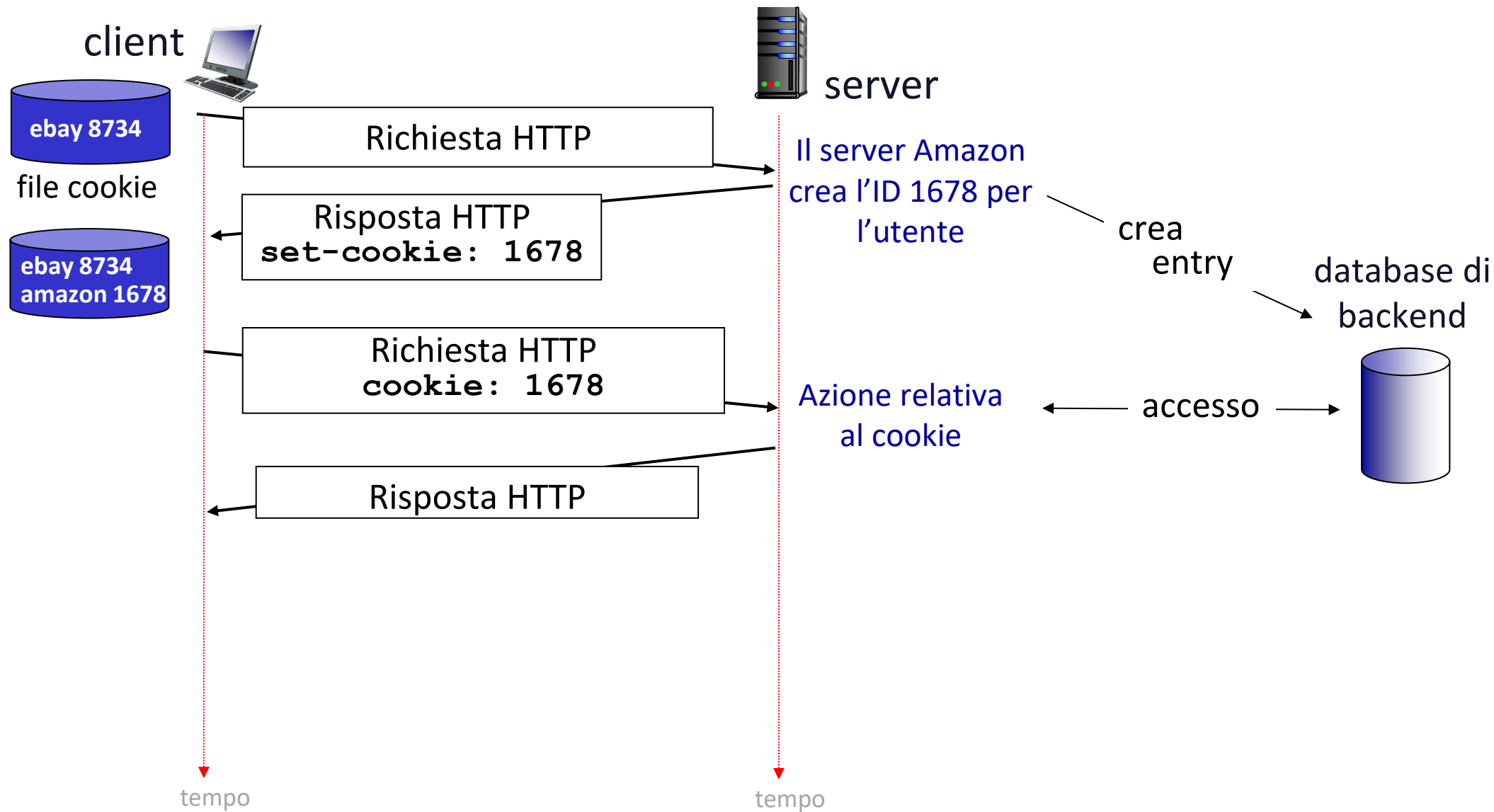
Mantenere lo stato in HTTP: i cookies



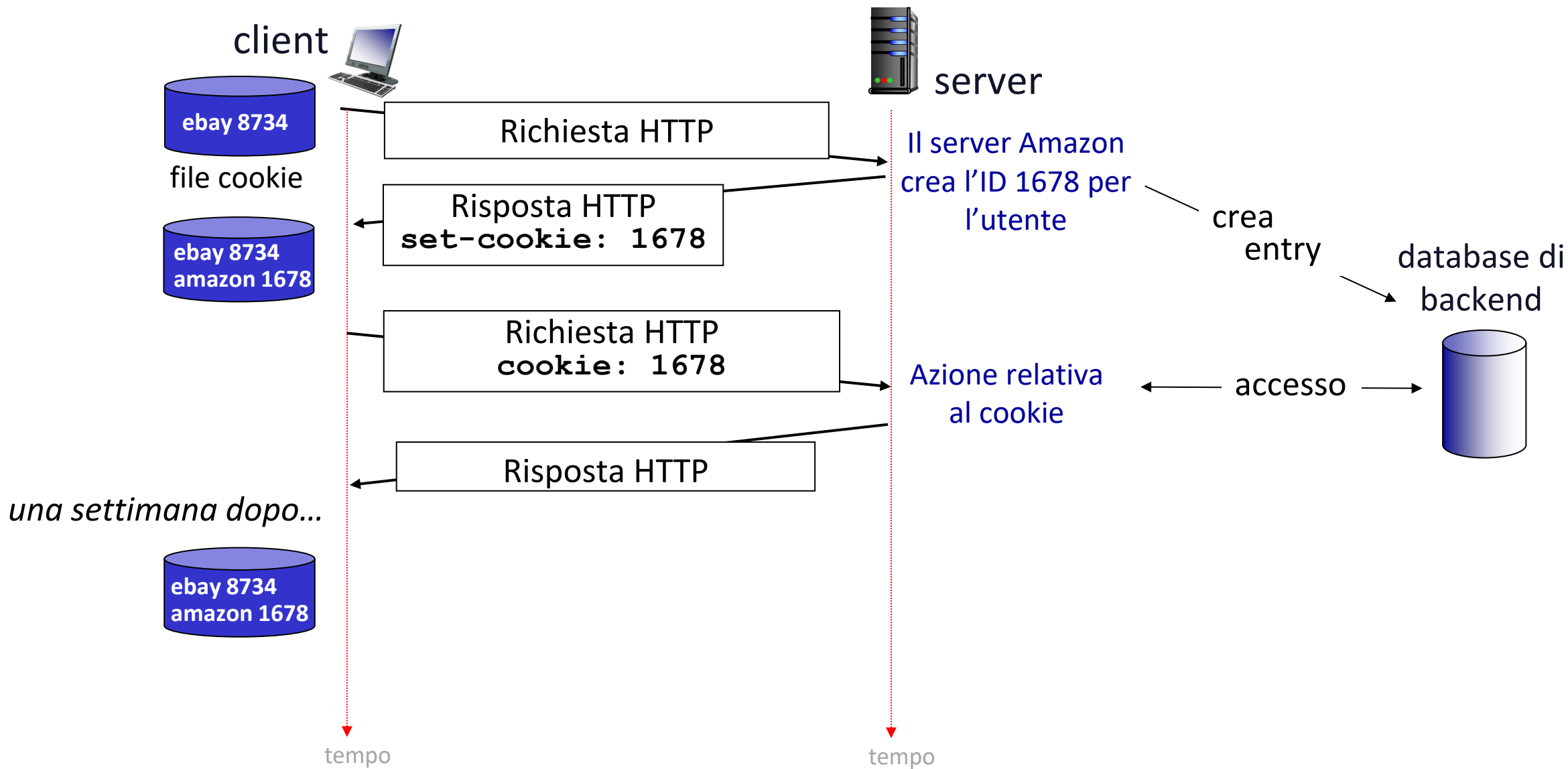
Mantenere lo stato in HTTP: i cookies



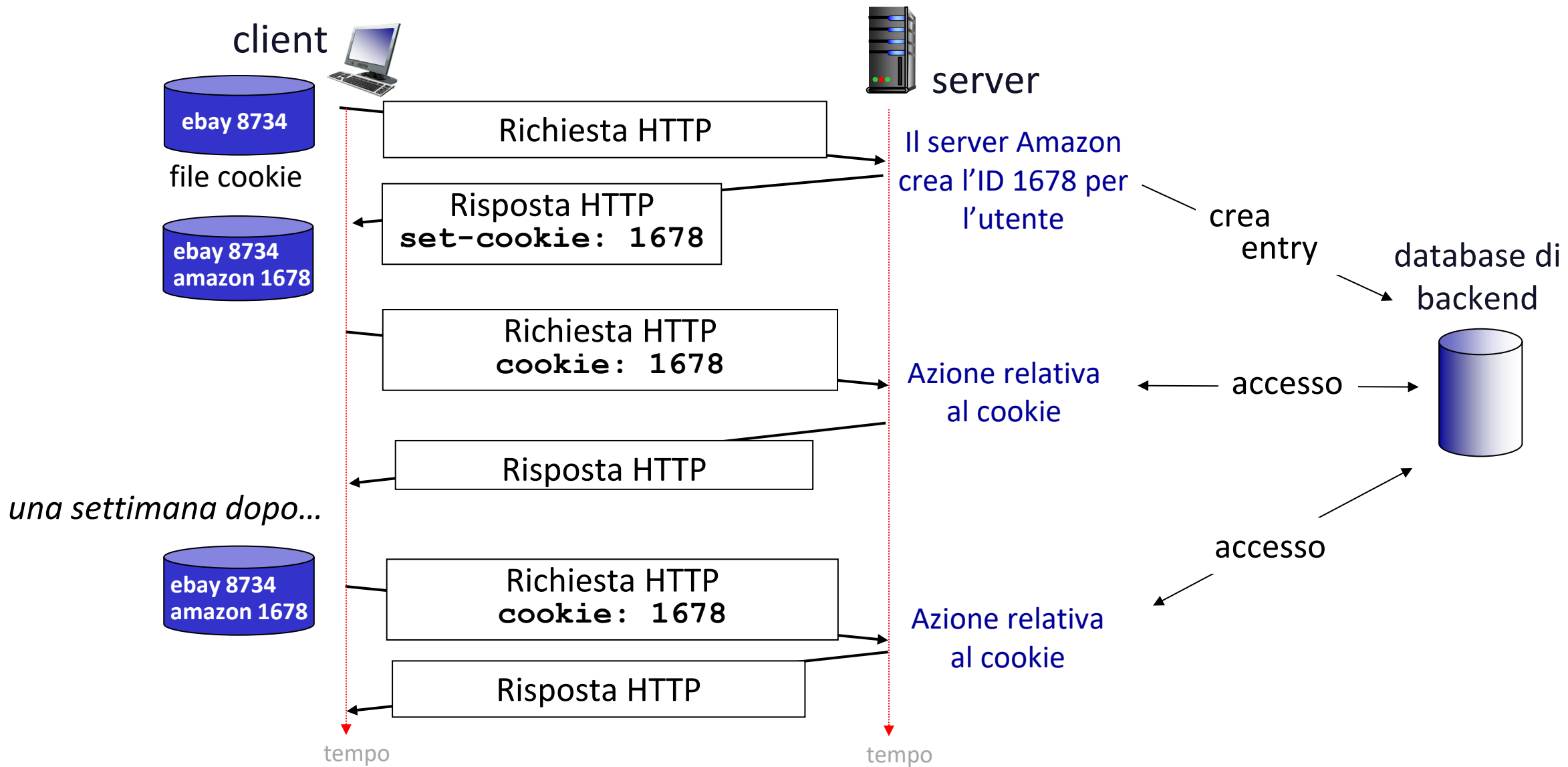
Mantenere lo stato in HTTP: i cookies



Mantenere lo stato in HTTP: i cookies



Mantenere lo stato in HTTP: i cookies



Web cache

- **Obiettivo:** soddisfare le richieste dei client per mezzo di **Web cache** (o **server replica**, o **server proxy**) disseminate localmente (es. nelle reti istituzionali o di accesso) e non per mezzo del **server Web di origine**
- Il browser invia tutte le richieste HTTP alla Web cache:
 - **Se** l'oggetto richiesto si trova in cache: l'oggetto viene inviato dalla cache
 - **Altrimenti** la cache richiede l'oggetto al server Web di origine, lo riceve, e lo invia al client



client

Web
cache



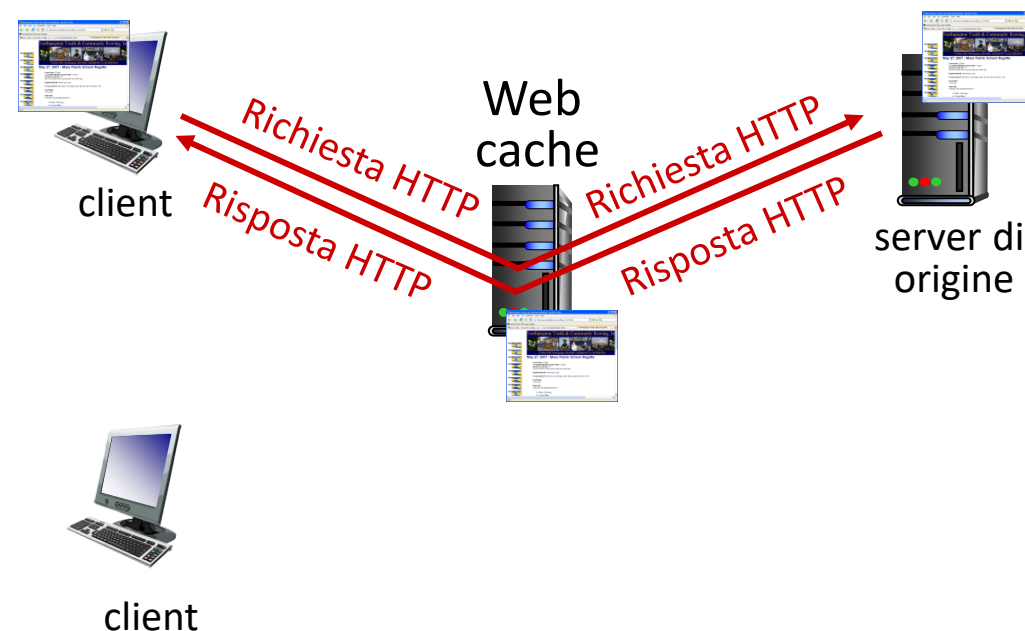
server di
origine



client

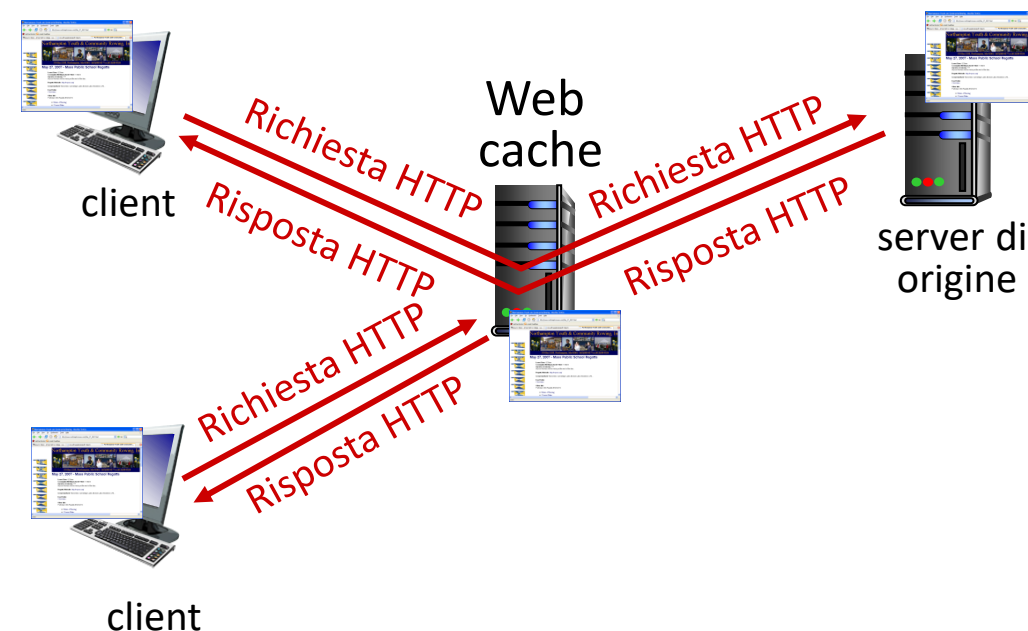
Web cache

- **Obiettivo:** soddisfare le richieste dei client per mezzo di **Web cache** (o **server replica**, o **server proxy**) disseminate localmente (es. nelle reti istituzionali o di accesso) e non per mezzo del **server Web di origine**
- Il browser invia tutte le richieste HTTP alla Web cache:
 - **Se** l'oggetto richiesto si trova in cache: l'oggetto viene inviato dalla cache
 - **Altrimenti** la cache richiede l'oggetto al server Web di origine, lo riceve, e lo invia al client



Web cache

- **Obiettivo:** soddisfare le richieste dei client per mezzo di **Web cache** (o **server replica**, o **server proxy**) disseminate localmente (es. nelle reti istituzionali o di accesso) e non per mezzo del **server Web di origine**
- Il browser invia tutte le richieste HTTP alla Web cache:
 - **Se** l'oggetto richiesto si trova in cache: l'oggetto viene inviato dalla cache
 - **Altrimenti** la cache richiede l'oggetto al server Web di origine, lo riceve, e lo invia al client



Web cache

- Una Web cache svolge un ruolo sia di client che di server
 - E' un server per le richieste HTTP da parte dei client
 - E' un client per il server di origine
- Il server di origine comunica alla Web cache se l'oggetto può essere soggetto a **caching** (e per quanto tempo) nell'intestazione della risposta HTTP

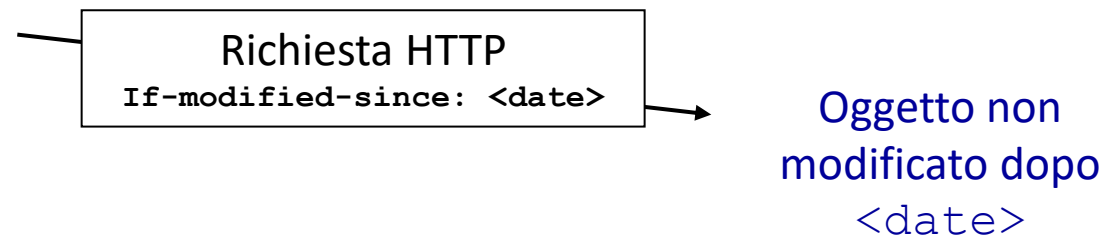
```
Cache-Control: max-age=<seconds>
```

```
Cache-Control: no-cache
```

- La rete Internet è disseminata di Web cache
- **Perché** effettuare Web caching?
 - Riduce il tempo di risposta per le richieste HTTP da parte dei client
 - La cache si trova **in prossimità** del client
 - Riduce il traffico in uscita dalle reti (es. reti istituzionali o di accesso)
 - Distributori di contenuti (**content provider**) con infrastrutture poco performanti possono recapitare i contenuti in modo più efficace

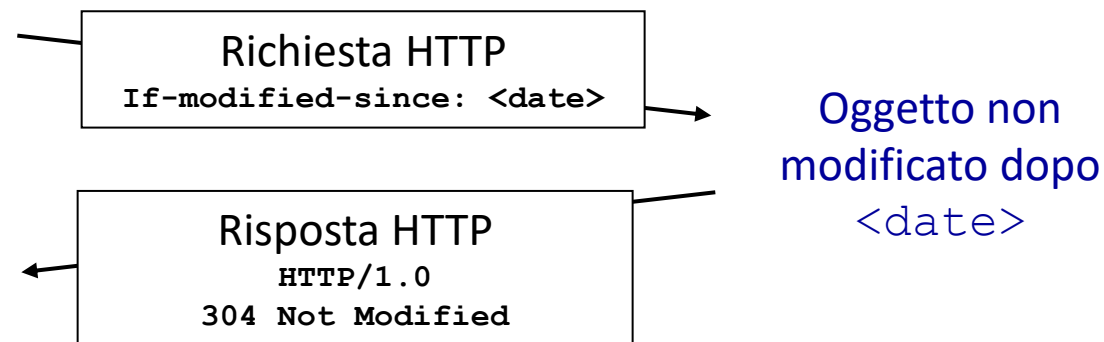
GET condizionale

- **Obiettivo:** non inviare un oggetto se una cache la sua versione già aggiornata
- **Client:** specifica la data della copia dell'oggetto in cache nella richiesta HTTP
If-modified-since: <date>
- **Server:** la risposta non include alcun oggetto se la copia dell'oggetto in cache è l'ultima versione:
HTTP/1.0 304 Not Modified
- **Nota:** anche i browser hanno una loro Web cache locale nella memoria del dispositivo!
 - No HTTP ma chiamate di sistema



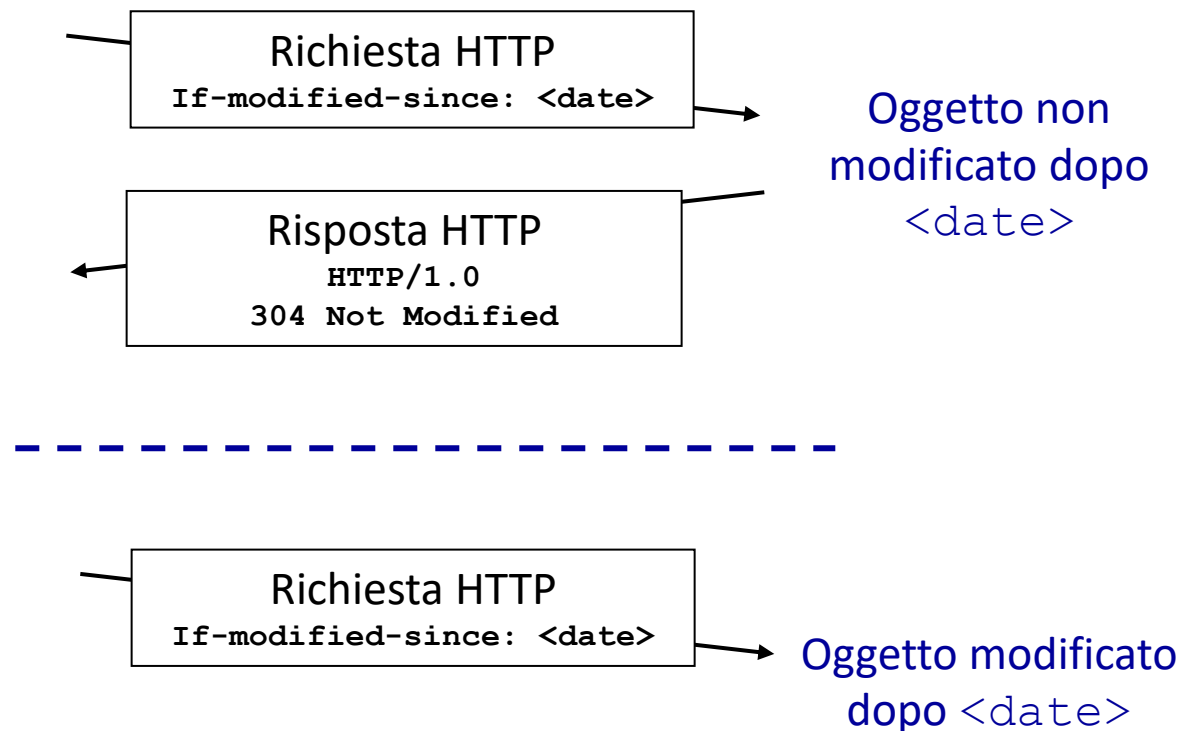
GET condizionale

- **Obiettivo:** non inviare un oggetto se una cache la sua versione già aggiornata
- **Client:** specifica la data della copia dell'oggetto in cache nella richiesta HTTP
If-modified-since: <date>
- **Server:** la risposta non include alcun oggetto se la copia dell'oggetto in cache è l'ultima versione:
HTTP/1.0 304 Not Modified
- **Nota:** anche i browser hanno una loro Web cache locale nella memoria del dispositivo!
 - No HTTP ma chiamate di sistema



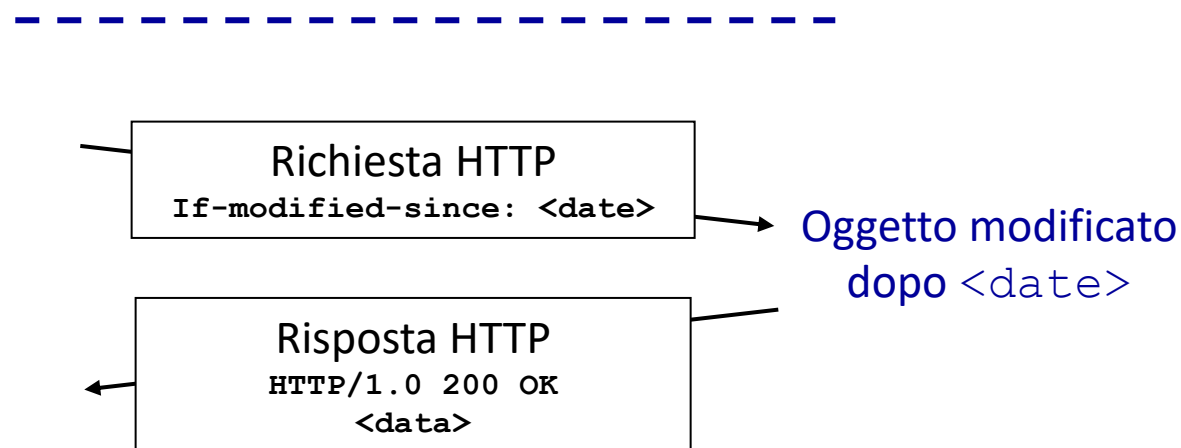
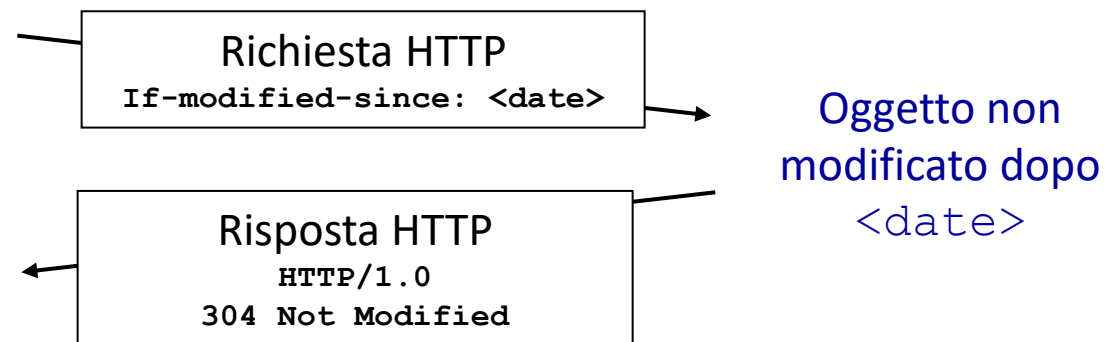
GET condizionale

- **Obiettivo:** non inviare un oggetto se una cache la sua versione già aggiornata
- **Client:** specifica la data della copia dell'oggetto in cache nella richiesta HTTP
If-modified-since: <date>
- **Server:** la risposta non include alcun oggetto se la copia dell'oggetto in cache è l'ultima versione:
HTTP/1.0 304 Not Modified
- **Nota:** anche i browser hanno una loro Web cache locale nella memoria del dispositivo!
 - No HTTP ma chiamate di sistema



GET condizionale

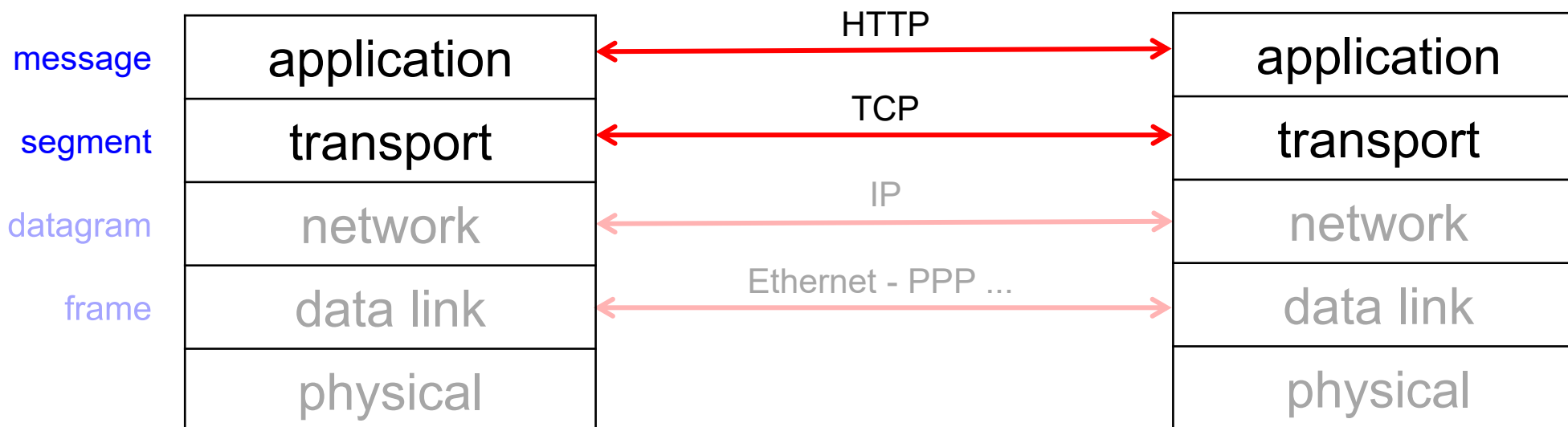
- **Obiettivo:** non inviare un oggetto se una cache la sua versione già aggiornata
- **Client:** specifica la data della copia dell'oggetto in cache nella richiesta HTTP
If-modified-since: <date>
- **Server:** la risposta non include alcun oggetto se la copia dell'oggetto in cache è l'ultima versione:
HTTP/1.0 304 Not Modified
- **Nota:** anche i browser hanno una loro Web cache locale nella memoria del dispositivo!
 - No HTTP ma chiamate di sistema





Message vs. stream communication: recap

Message vs. stream communication



- Il livello di applicazione invia **messaggi**
 - Comunicazione orientata al messaggio (message-oriented communication)
- Tali messaggi per il livello di trasporto sono, nel loro complesso, dei **flussi di byte** (stream) da trasportare in rete
 - Comunicazione orientata al flusso (stream-oriented communication)

Message vs. stream communication

Applicazione:

- Invia i messaggi al servizio di trasporto come stream di byte
- Legge lo stream di byte dal livello di trasporto e ricostruisce i messaggi

```
GET /index.html HTTP/1.1<CR><LF>Host: www.unimib.it<CR><LF>User-agent: Mozilla/4.0<CR><LF><CR><LF>
```

Message vs. stream communication

Applicazione:

- Invia i messaggi al servizio di trasporto come stream di byte
- Legge lo stream di byte dal livello di trasporto e ricostruisce i messaggi

```
GET /index.html HTTP/1.1<CR><LF>Host: www.unimib.it<CR><LF>User-agent: Mozilla/4.0<CR><LF><CR><LF>
```

Servizio UDP:

- Scompone lo stream di byte ricevuto in segmenti
- Invia i segmenti al livello di rete (modalità best-effort)

```
GET /index.html HTTP /1.1<CR><LF>Host: ww w.unimib.it<CR><LF>U ser-agent: Mozilla/4 .0<CR><LF><CR><LF>
```

Message vs. stream communication

Applicazione:

- Invia i messaggi al servizio di trasporto come stream di byte
- Legge lo stream di byte dal livello di trasporto e ricostruisce i messaggi

```
GET /index.html HTTP/1.1<CR><LF>Host: www.unimib.it<CR><LF>User-agent: Mozilla/4.0<CR><LF><CR><LF>
```

Servizio UDP:

- Scompone lo stream di byte ricevuto in segmenti
- Invia i segmenti al livello di rete (modalità best-effort)

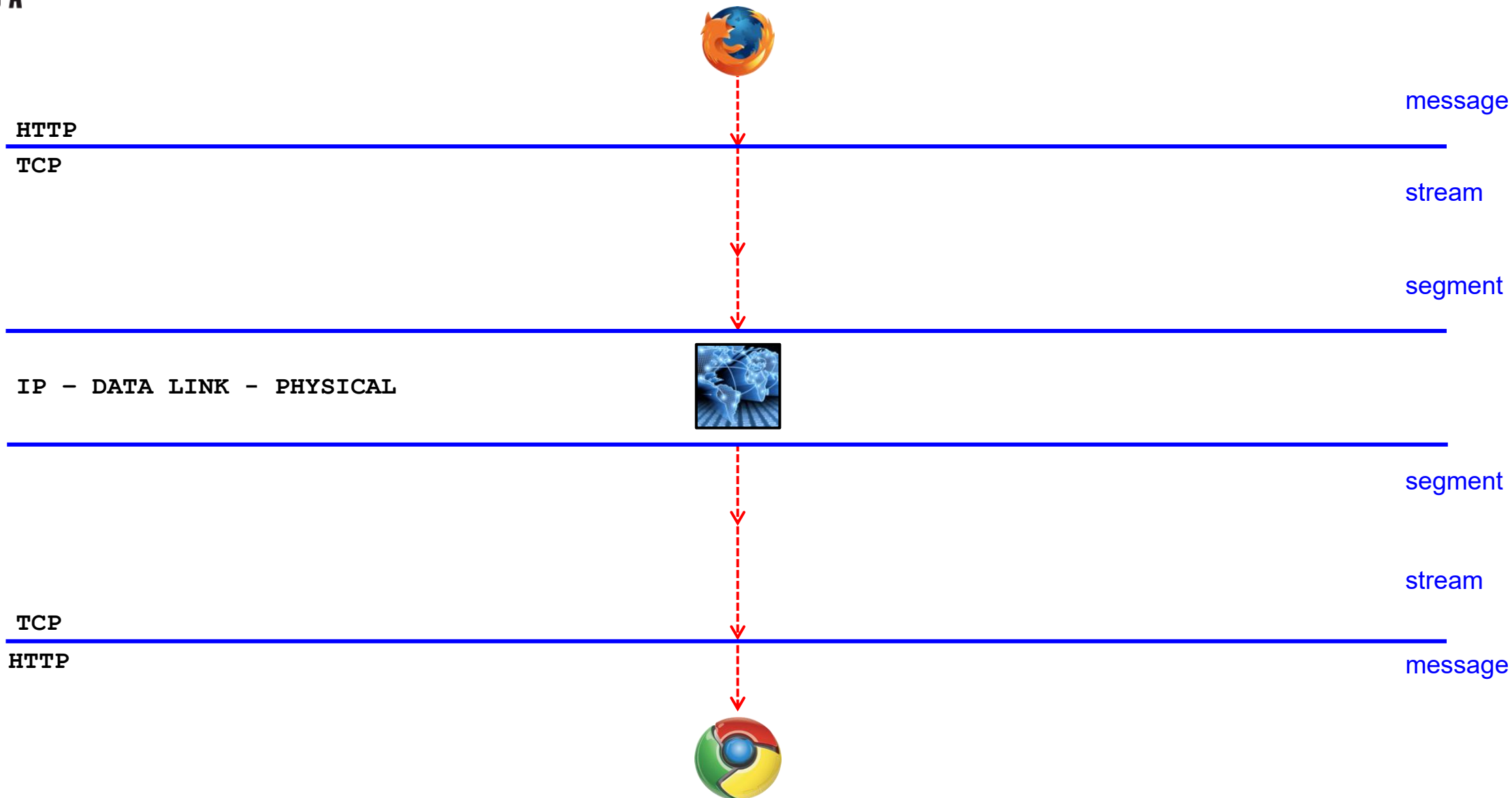
```
GET /index.html HTTP /1.1<CR><LF>Host: ww w.unimib.it<CR><LF>U ser-agent: Mozilla/4 .0<CR><LF><CR><LF>
```

Servizio TCP:

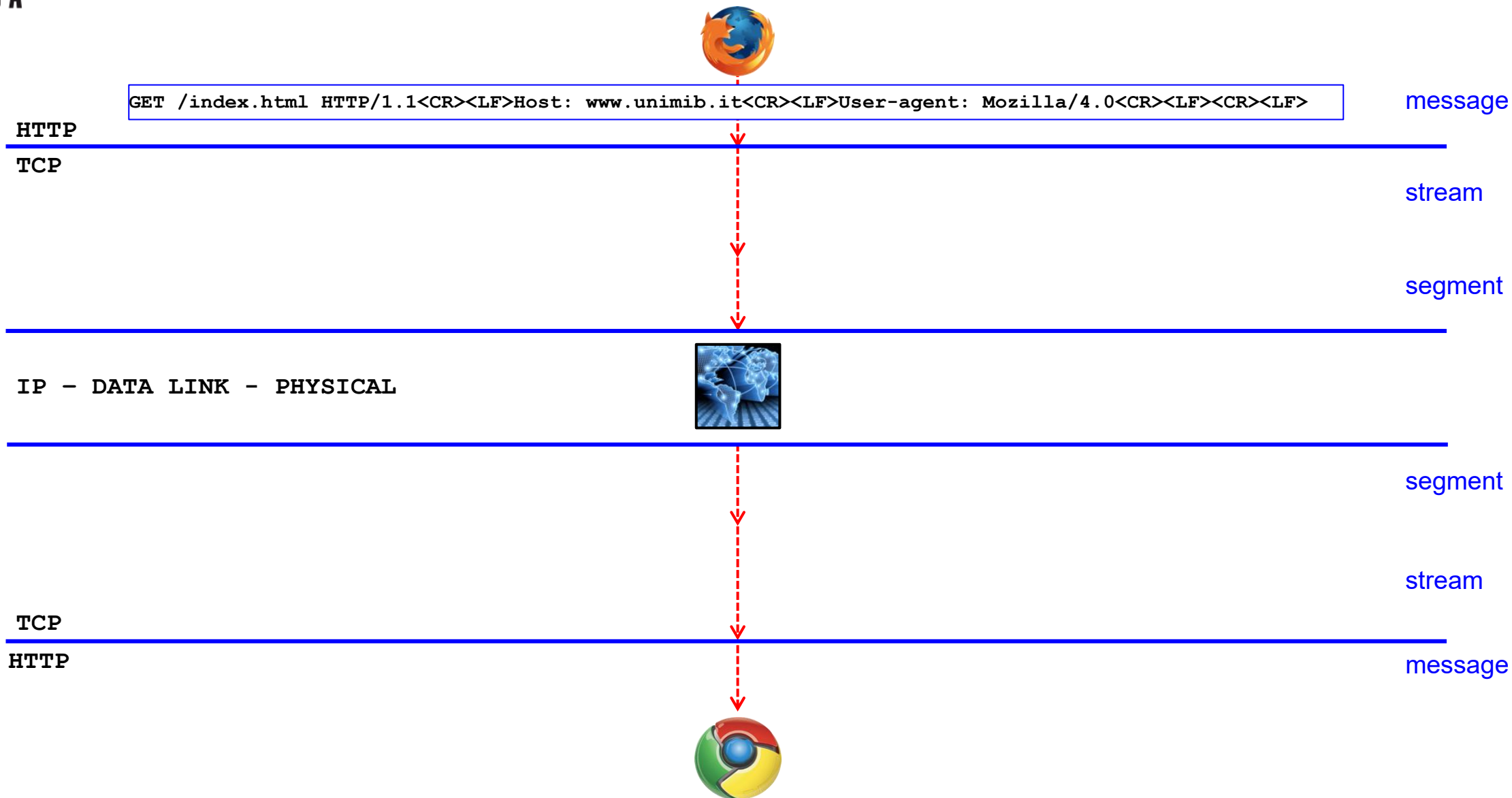
- Scompone e invia segmenti come UDP
- Ogni segmento viene numerato per garantire:
 - Riordinamento dei segmenti arrivati
 - Controllo delle duplicazioni (scarto dei segmenti con stesso numero di sequenza)
 - Controllo delle perdite (rinvio dei segmenti mancanti)

```
0GET /index.html HTTP 1w.unimib.it<CR><LF>U 2/1.1<CR><LF>Host: ww 3ser-agent: Mozilla/4 4.0<CR><LF><CR><LF>
```

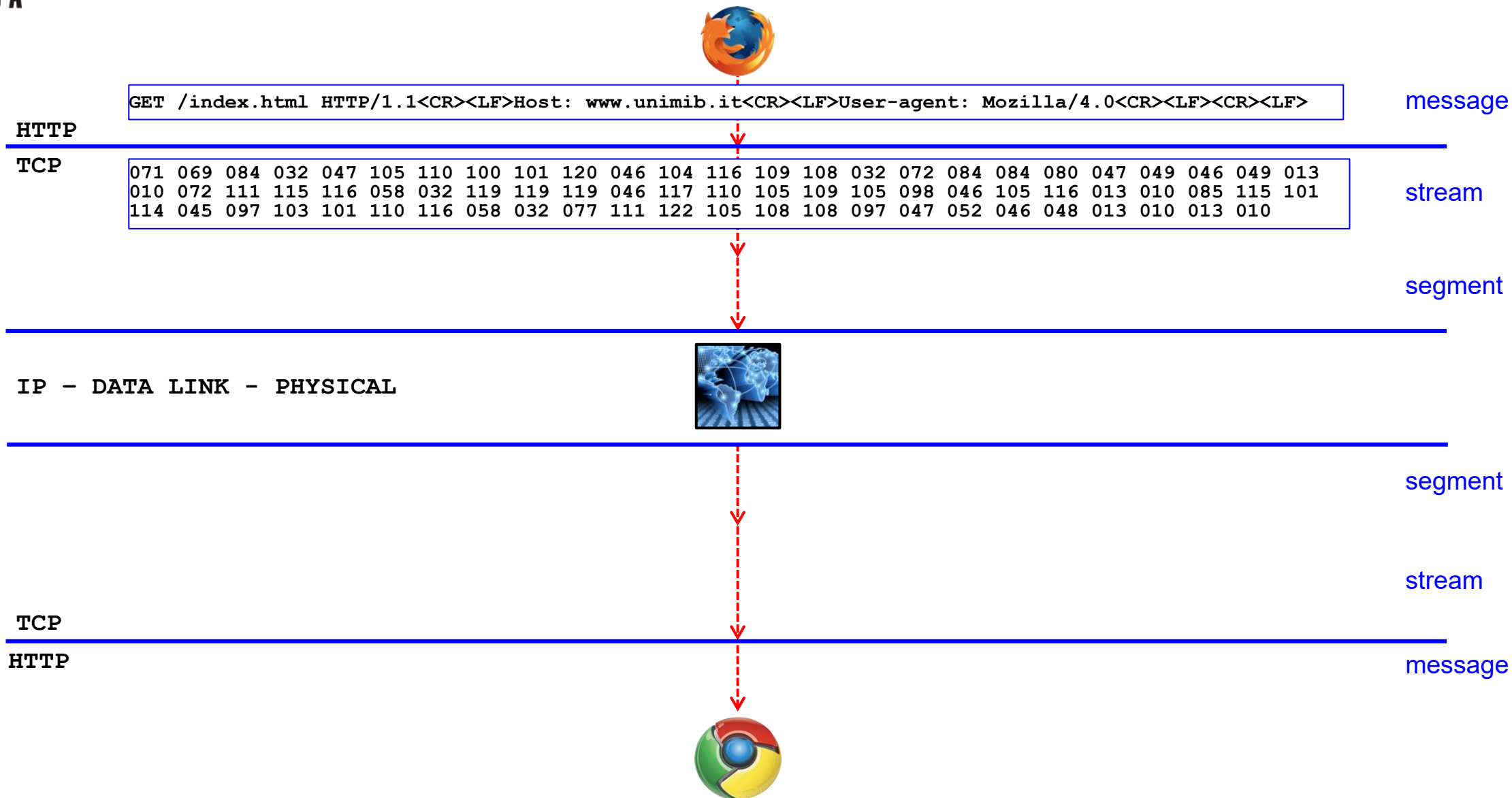
Message vs. stream communication



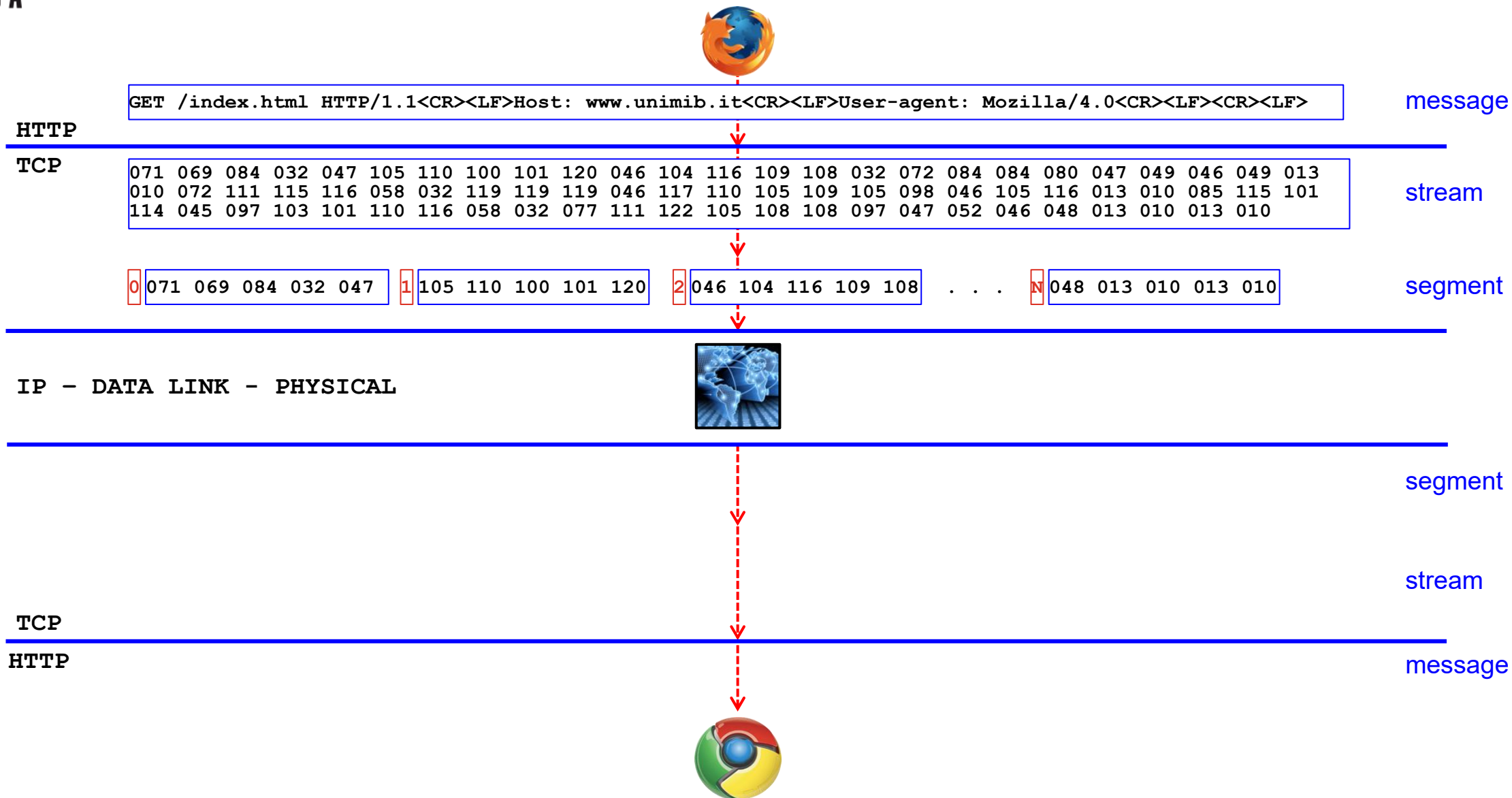
Message vs. stream communication



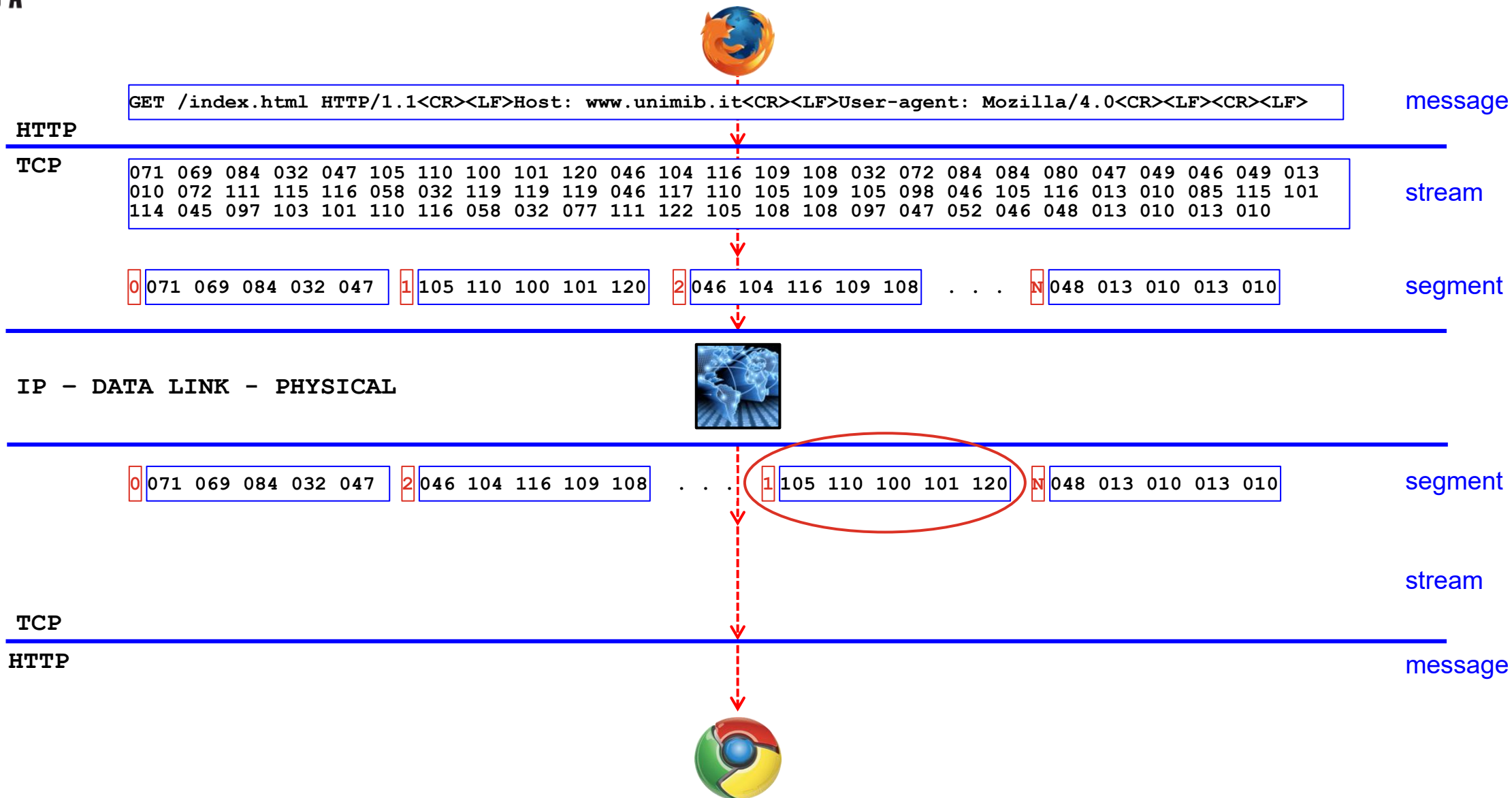
Message vs. stream communication



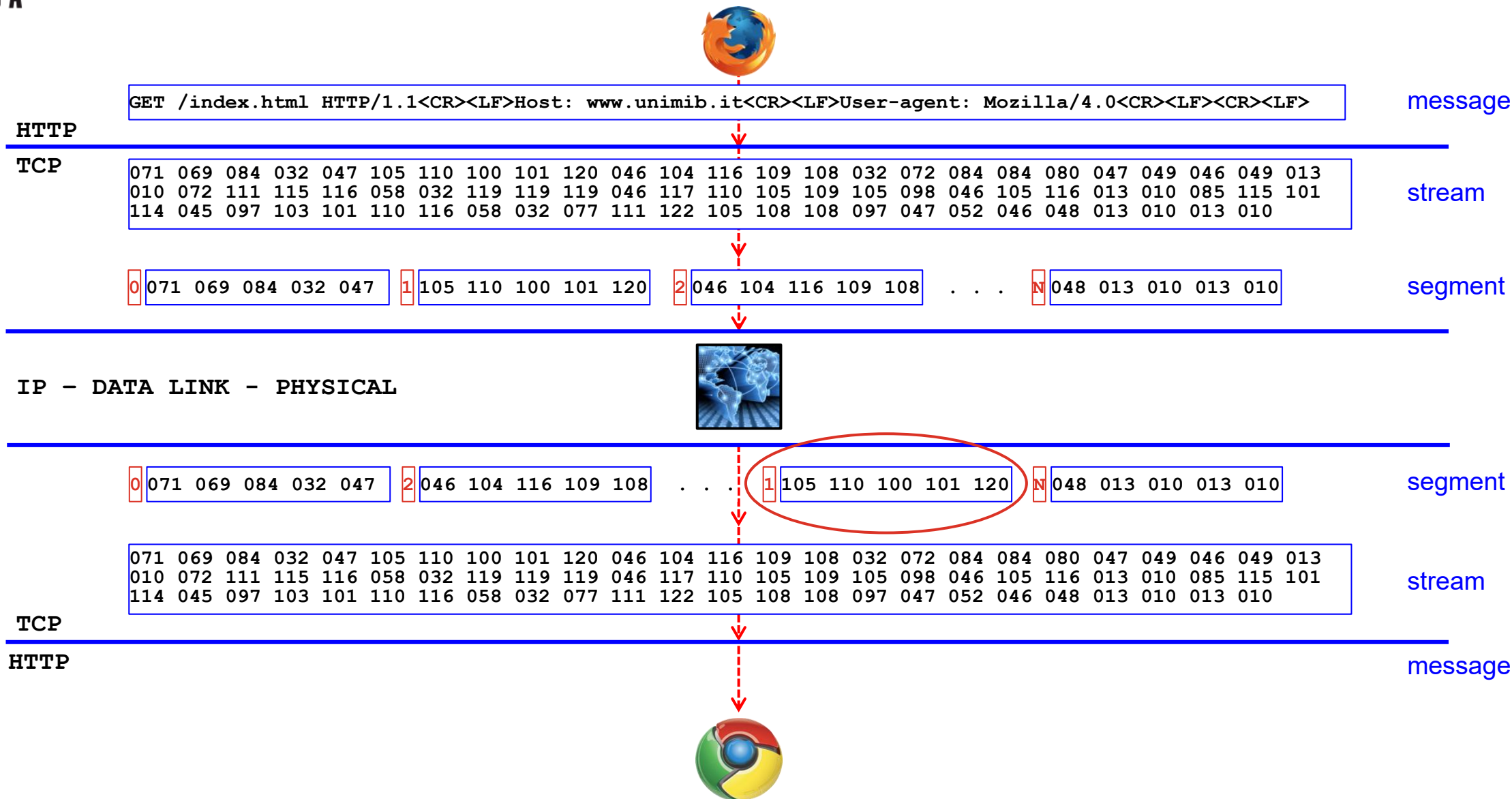
Message vs. stream communication



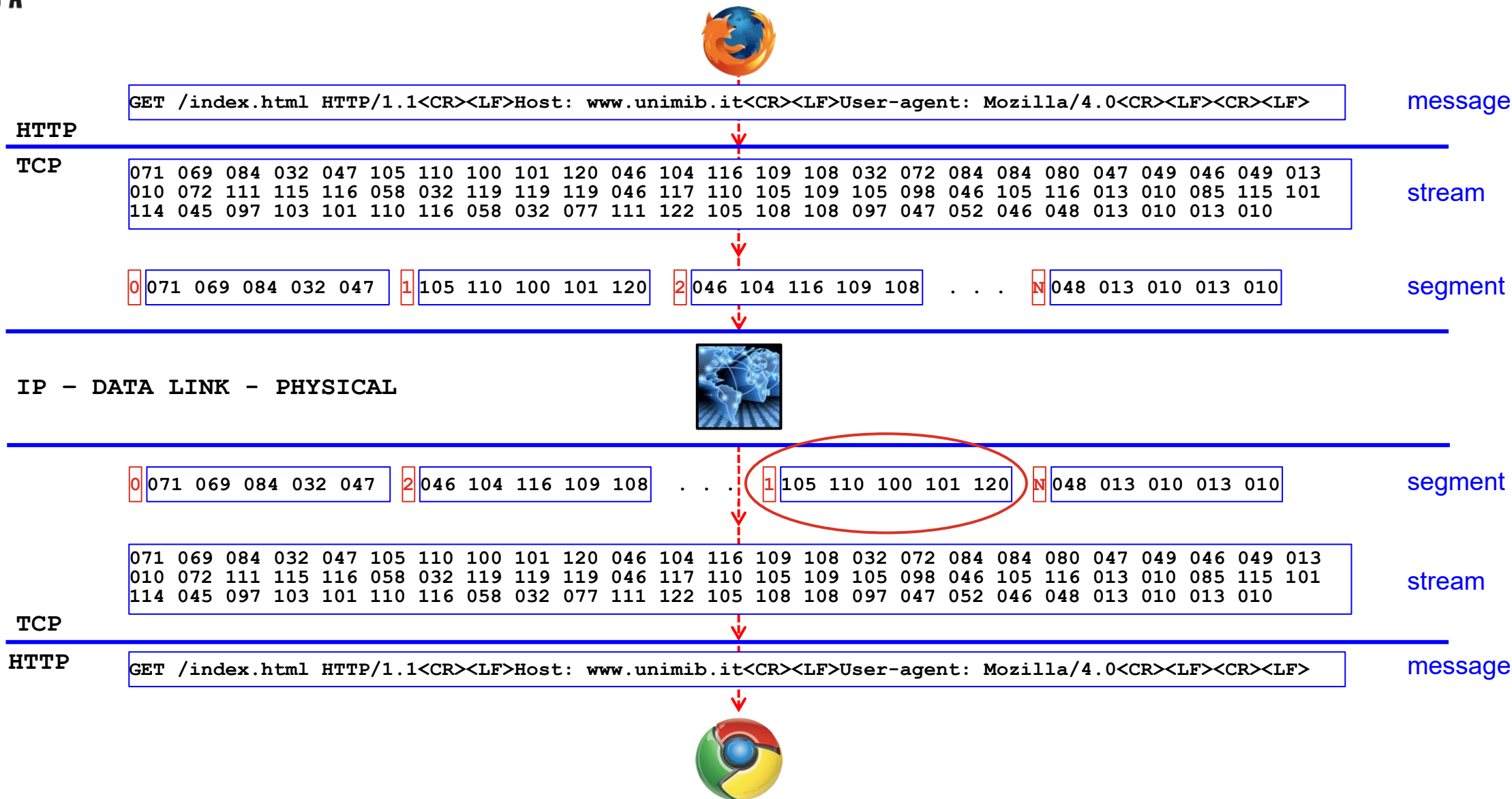
Message vs. stream communication



Message vs. stream communication



Message vs. stream communication

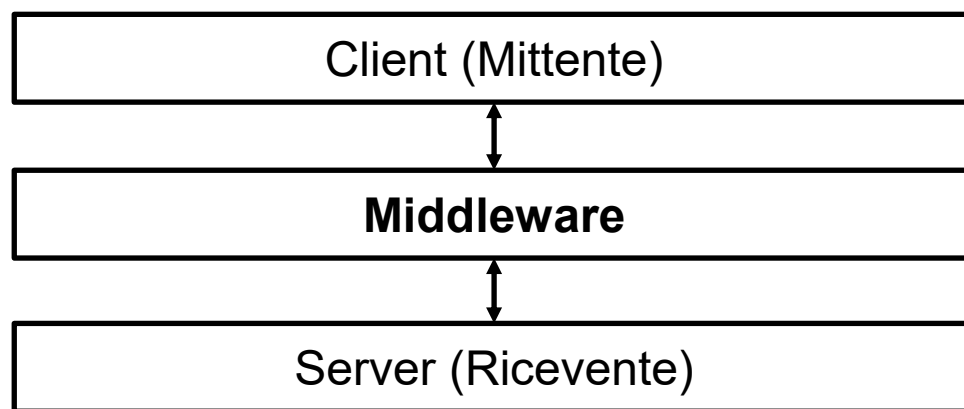




Message-oriented communication: tipi di comunicazione

Tipi di comunicazione

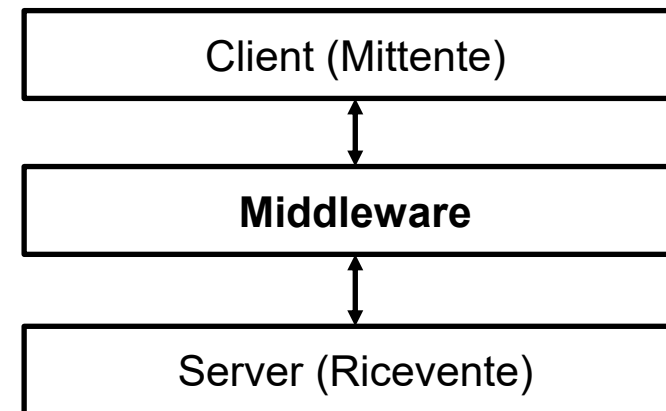
- **Focus:** quali sono i possibili tipi di comunicazione quando un **middleware** è coinvolto nella comunicazione
 - Nel paradigma client-server, il middleware può essere visto come un **servizio aggiuntivo** che si frappone tra client (mittente) e server (ricevente)
 - Implementazione tipica: due software (lato client e lato server) che comunicano tra loro



- **Esempio:** scambio di email
 - Il middleware è il sistema di consegna email (**mail delivery system**)

Tipi di comunicazione

- Classificazione: due categorie ortogonali
 - Comunicazione **sincrona** vs. **asincrona**
 - Comunicazione **persistente** vs. **transiente** (o transitoria)



■ Comunicazione **sincrona**:

- Il client sospende l'esecuzione finché non sa che il suo messaggio (o richiesta) è recapitato al server

■ Comunicazione **asincrona**:

- Il client continua l'esecuzione non appena il messaggio è reso disponibile al middleware per la trasmissione

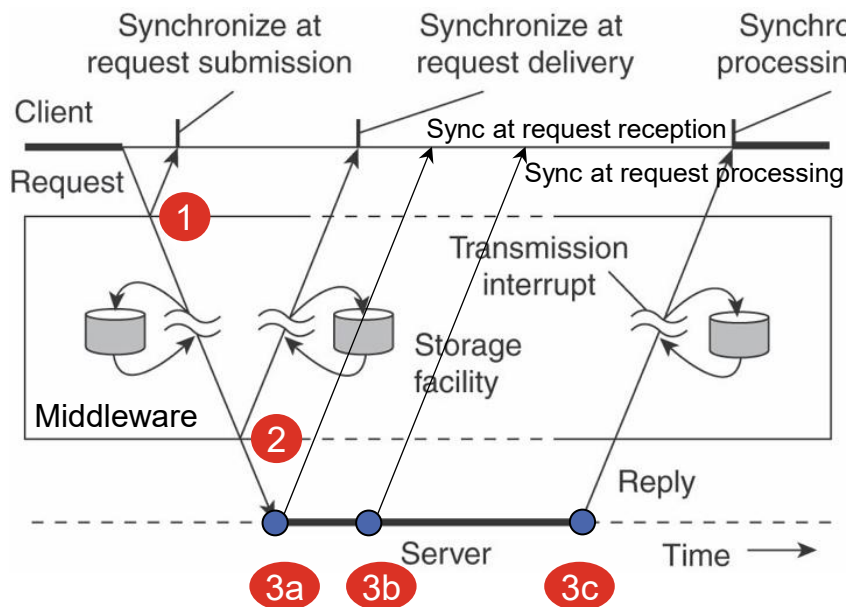
■ Comunicazione **persistente**:

- Un messaggio viene immagazzinato dal middleware (almeno) per il tempo necessario alla sua consegna
 - Non è necessario che client e server siano contemporaneamente in esecuzione

■ Comunicazione **transiente**:

- Un messaggio viene inoltrato dal middleware solo se sia il client che il server sono contemporaneamente in esecuzione, altrimenti è scartato

Punti di sincronizzazione



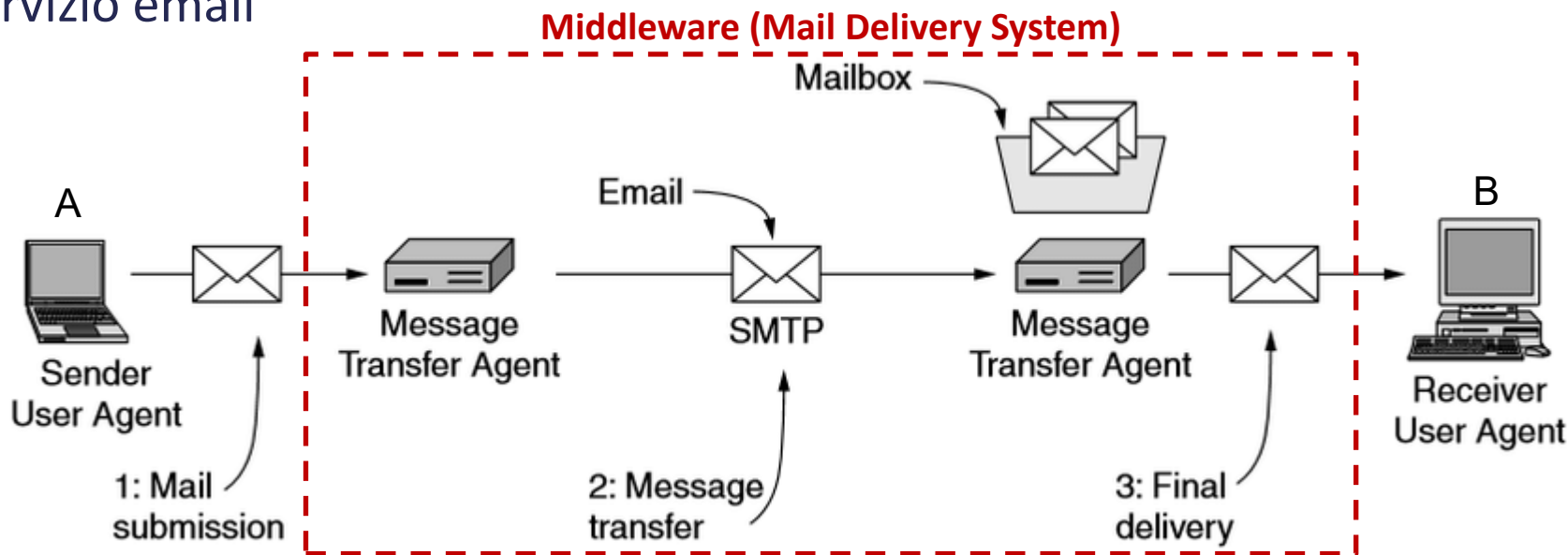
Punti di sincronizzazione:

- 1 **All'invio** della richiesta (request submission)
 - 2 **All'inoltro** della richiesta (request delivery)
 - 3a **Alla ricezione** della richiesta (request reception)
 - 3b **All'elaborazione** della richiesta (at request processing)
 - 3c **Dopo l'elaborazione** della richiesta (after processing)
- } Middleware
 } Server

1. Il client sospende l'esecuzione finché il software middleware lato client non notifica che si è preso in carico l'inoltro della richiesta
2. Il client sospende l'esecuzione finché la richiesta non è inoltrata ed è notificato che è stata recapitata al software middleware lato server
3. Il client sospende l'esecuzione finché la richiesta
 - a. E' stata ricevuta dal server e l'evento è stato notificato
 - b. E' in fase di elaborazione da parte del server e l'evento è stato notificato
 - c. E' stata completamente elaborata dal server e l'evento è stato notificato

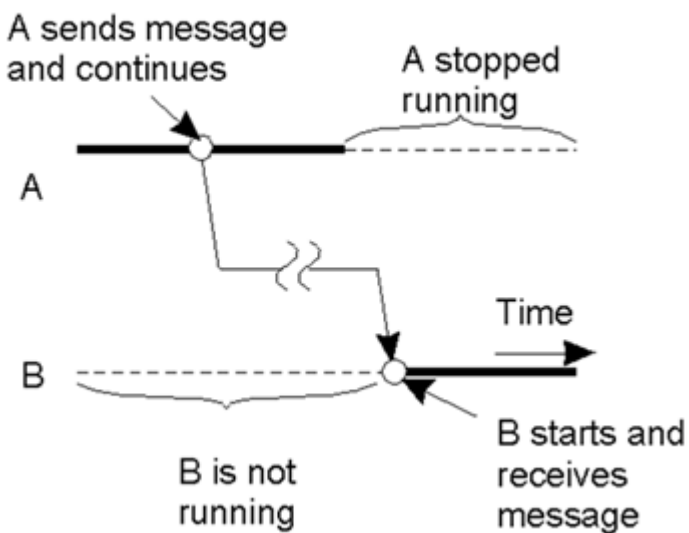
Tipi di comunicazione e punti di sincronizzazione

- **Attenzione!** “Comunicazione sincrona” e “comunicazione asincrona” si riferiscono all’interazione tra mittente e ricevente!
- **Esempio:** servizio email



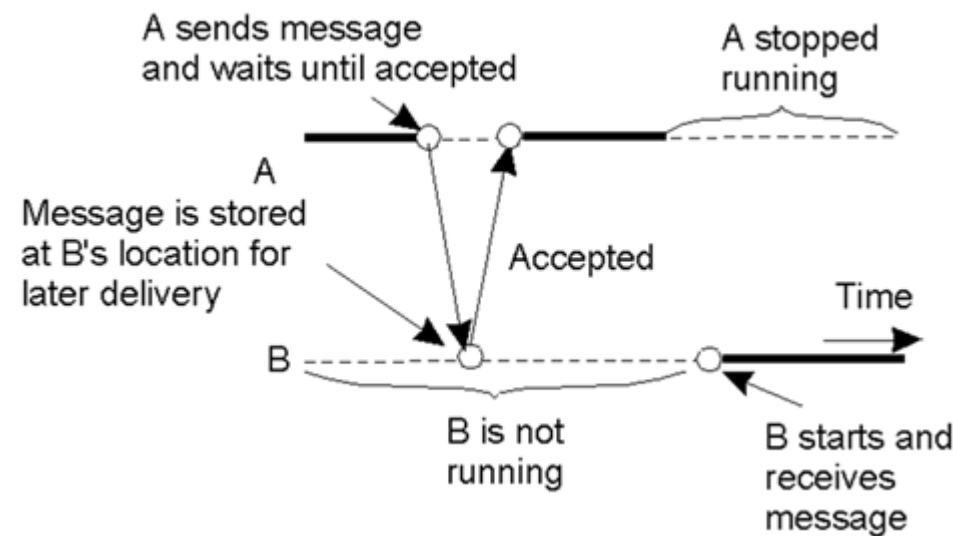
- E' un tipico esempio di **sistema persistente asincrono**
- Si ha un punto di sincronizzazione **all'invio della richiesta**
 - E' con il middleware lato mittente, **non** con il ricevente!
- Il mittente sospende l'esecuzione finché il server SMTP (Message Transfer Agent) del mittente non ha ricevuto il messaggio

Comunicazione persistente: modelli



■ Comunicazione persistente asincrona

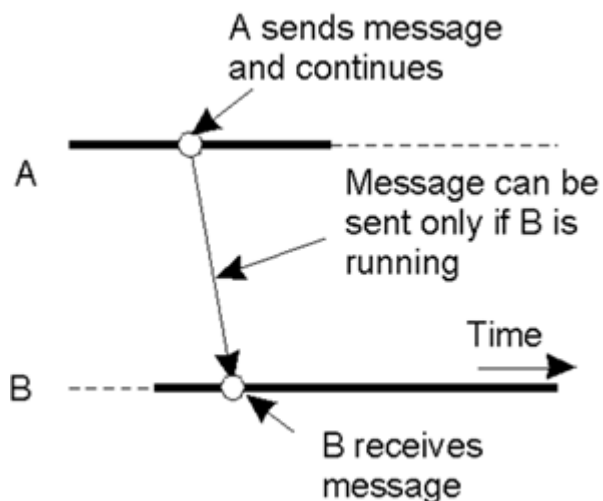
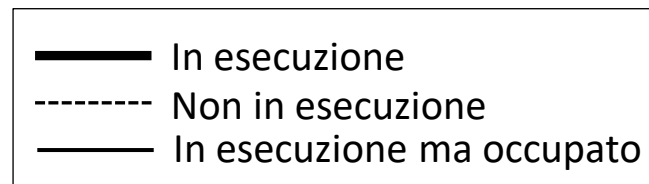
- Punto di sincronizzazione: all'invio della richiesta (1)
- Nella pratica: A continua l'esecuzione non appena il software middleware lato mittente ha ricevuto il messaggio e l'ha notificato
- Esempio: servizio email



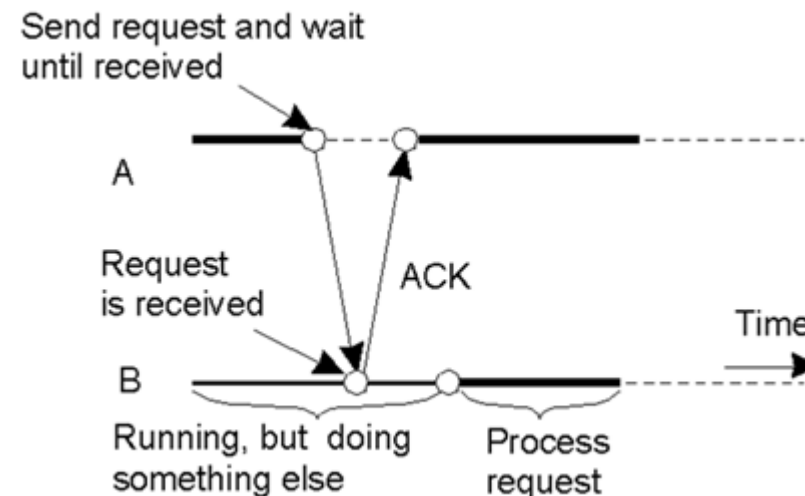
■ Comunicazione persistente sincrona

- Punto di sincronizzazione: all'inoltro della richiesta (2)
- Nella pratica: A continua l'esecuzione quando il software middleware lato ricevente ha ricevuto il messaggio e l'ha notificato al mittente
- Esempio: transazione bancaria con POS

Comunicazione transiente: modelli (1)

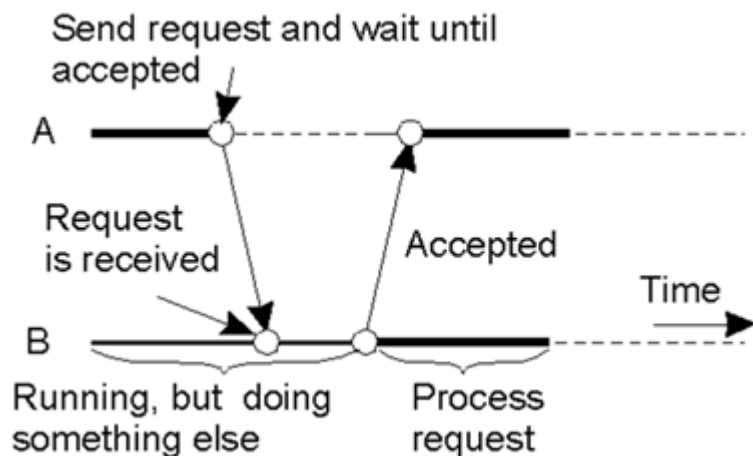


- **Comunicazione transiente asincrona**
 - Punto di sincronizzazione: all'invio della richiesta (1)
- Nella pratica: A continua l'esecuzione quanto il software middleware lato mittente ha ricevuto il messaggio, che viene recapitato solo se il ricevente *B* è in esecuzione
- Esempio: UDP



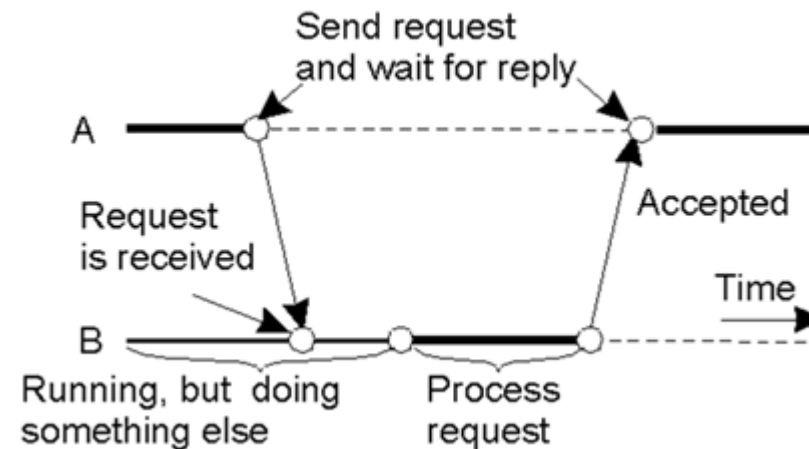
- **Comunicazione transiente sincrona basata sulla ricevuta (receipt-based)**
 - Punto di sincronizzazione: alla ricezione della richiesta (3a)
- Nella pratica: A continua l'esecuzione quando il ricevente *B*, che è in esecuzione, notifica la ricezione con un ACK
- Esempi: TCP; singola spunta di Whatsapp nel protocollo di invio

Comunicazione transiente: modelli (2)



- **Comunicazione transiente sincrona basata sulla consegna (delivery-based)**

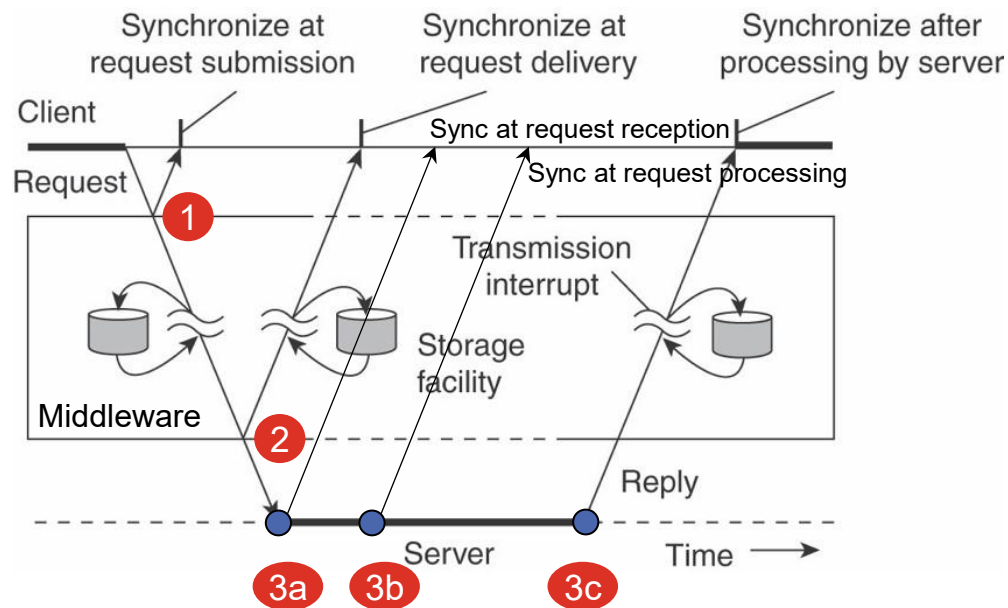
- Punto di sincronizzazione: all'elaborazione della richiesta (3b)
- Nella pratica: A continua l'esecuzione quando B, che è in esecuzione, notifica di aver iniziato a elaborare la richiesta
- Esempio: invio comandi a sever remoto (SSH)



- **Comunicazione transiente sincrona basata sulla risposta (response based)**

- Punto di sincronizzazione: dopo l'elaborazione della richiesta (3c)
- Nella pratica: A continua l'esecuzione quando B, che è in esecuzione, notifica di aver elaborato correttamente la richiesta
- Esempio: HTTP

Modelli di comunicazione e punti di sincronizzazione - Recap



Punti di sincronizzazione:

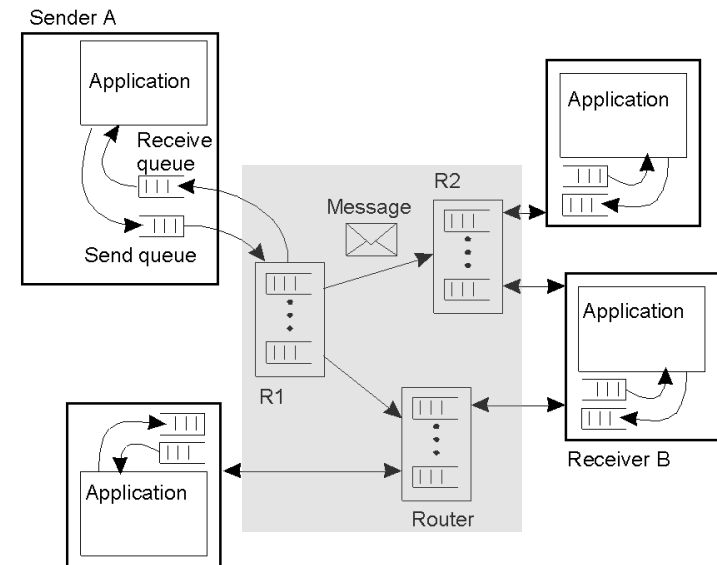
- 1 All'invio della richiesta → Comunicazione **persistente asincrona** o **transiente asincrona**
 - 2 All'inoltro della richiesta → Comunicazione **persistente sincrona**
 - 3a Alla ricezione della richiesta → Comunicazione **transiente sincrona basata sulla ricevuta**
 - 3b All'elaborazione della richiesta (at request processing) → Comunicazione **transiente sincrona basata sulla consegna**
 - 3c Dopo l'elaborazione della richiesta (after processing) → Comunicazione **transiente sincrona basata sulla risposta**
- } Middleware
 } Server



Message-oriented persistent communication

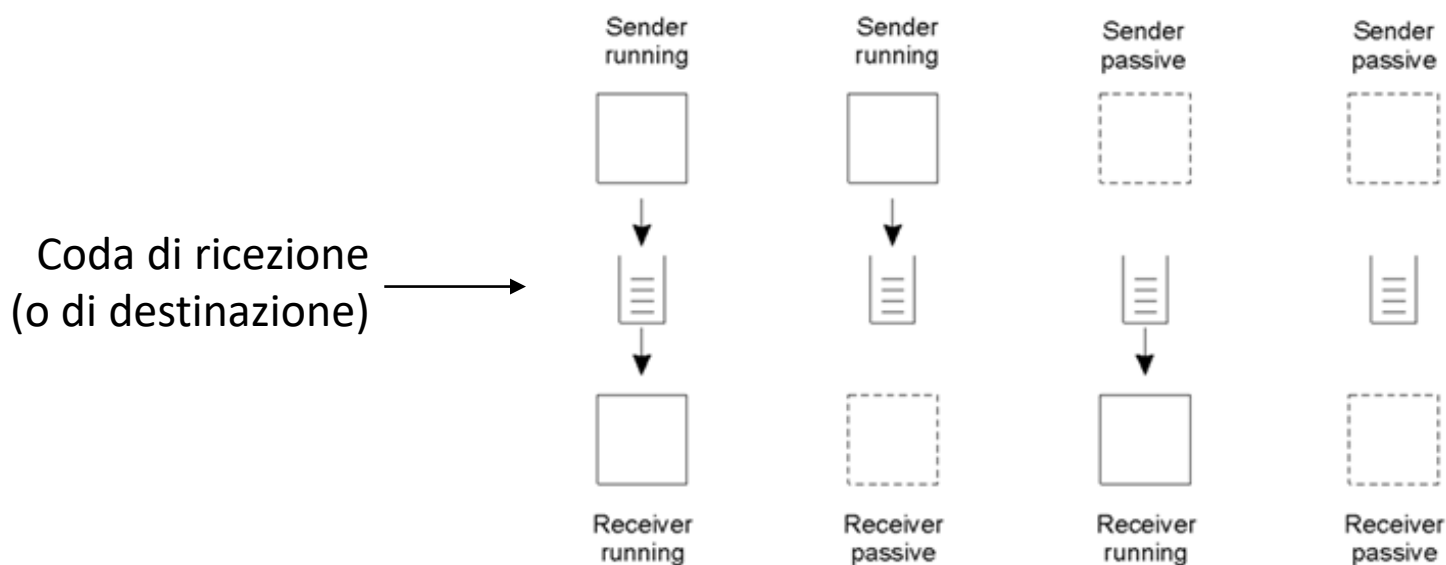
Modello a code di messaggi (message-queuing model)

- E' il pilastro della comunicazione **persistente asincrona**
- I sistemi basati su un modello a code di messaggio prendono anche il nome di **message-oriented middleware (MOM)**
- Caratteristiche fondamentali:
 - Capacità di storage per i messaggi (persistenza)
 - Disaccoppiamento temporale (asincronismo)
 - Orizzonti temporali laschi per la consegna (minuti, o anche più)
- Idea di base: le applicazioni comunicano inserendo i messaggi in code specifiche
 - Il modello prevede l'instradamento tra server di comunicazione (router) interconnessi
 - Nella pratica: solitamente server direttamente connessi
 - Il mittente ha di solito la garanzia esclusivamente che il messaggio sia inserito nella coda del ricevente



Modello a code di messaggi

- Disaccoppiamento temporale:
 - **Non** è necessario che il ricevente sia in esecuzione quando un messaggio è inserito nella sua coda di ricezione
 - **Non** è necessario che il mittente sia in esecuzione quando il messaggio è prelevato dal ricevente dalla sua coda di ricezione
- Una volta che un messaggio è inserito in una coda vi rimane finché non è rimosso
- Quattro combinazioni di stato possibili (e accettabili):



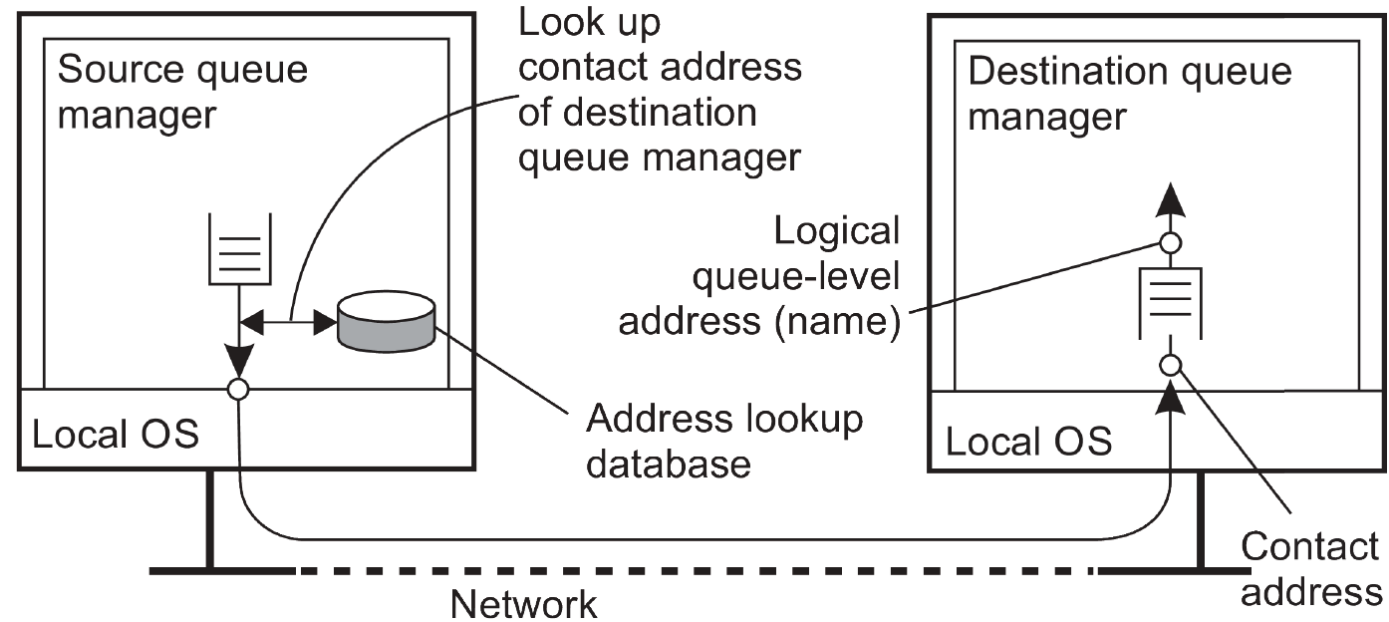
Modello a code di messaggi - Interfaccia di base

- Un messaggio può contenere qualsiasi dato
- L'**indirizzamento** prevede un identificatore univoco per ogni coda di ricezione
- L'**interfaccia di comunicazione** di base prevede quattro primitive

Primitiva	Significato	Tipo
PUT	Inserisci un messaggio in una determinata coda di invio (mittente)	Non bloccante
GET	Rimuovi il primo messaggio dalla coda di ricezione (ricevente)	Bloccante
POLL	Controlla una la presenza di messaggi in una coda e rimuovi il primo (ricevente)	Non bloccante
NOTIFY	Installa un handler (funzione di callback) che è chiamato dal middleware quando un messaggio è inserito in coda (middleware)	Non bloccante

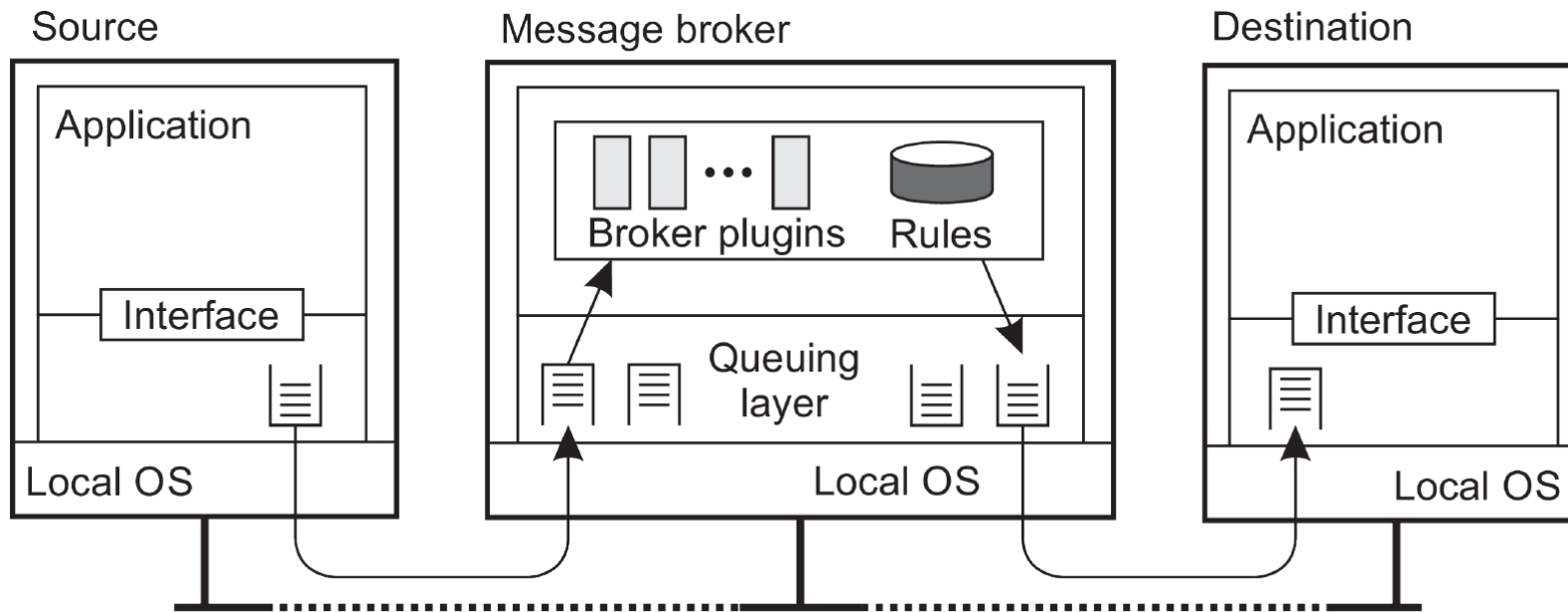
- Due modelli per la gestione di NOTIFY
 - **Event-driven**: il messaggio NOTIFY include anche il dato (GET non necessaria)
 - **Alert-only** (oggi meno diffuso): necessaria una GET (non bloccante) per recuperare il dato
- **Nota**: una funzione di callback può anche avviare un processo per il prelievo del dato!

Modello a code di messaggi - Architettura generale



- Le code sono gestate da **gestori di coda** (queue manager)
- L'applicazione mittente inserisce i messaggi nella sua **coda locale** specificando un **indirizzo logico** per la coda di destinazione
- Il gestore di coda si preoccupa poi di instradare il messaggio verso la coda di ricezione sulla base di un **indirizzo di contatto** (indirizzo del gestore di coda di destinazione)
 - L'associazione tra indirizzi logici e indirizzi di contatto è mantenuta in un **address lookup database**

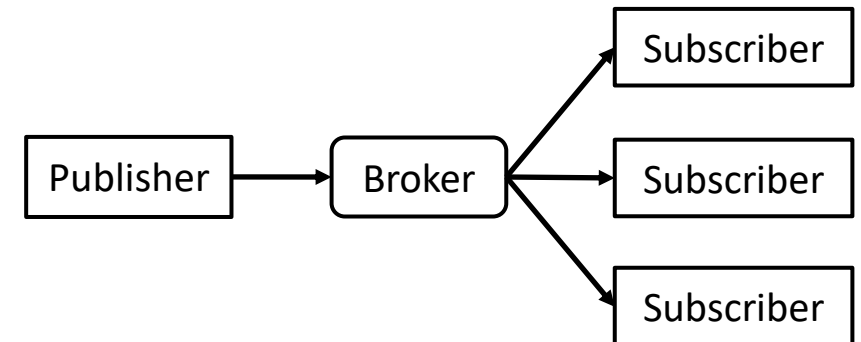
Modello a code di messaggi - Message broker



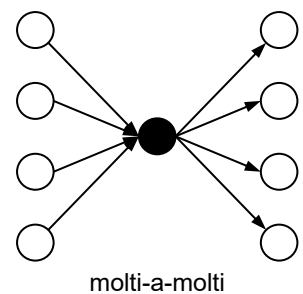
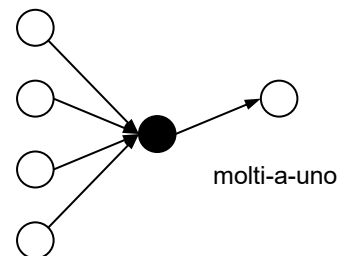
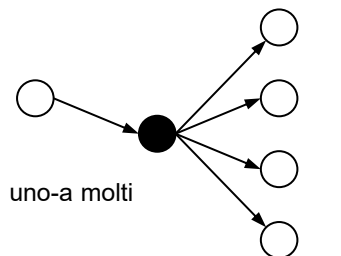
- I sistemi a code di messaggi assumono l'esistenza di un **protocollo di messaggistica comune** tra tutti i possibili mittenti e destinatari
 - Assunzione forte: la realtà è molto variegata (protocolli con sintassi e semantica differente)
- **Message broker:** nodo di conversione dei messaggi relativi a protocolli differenti
- Esempio di message broker: Apache Kafka

Message broker - Pattern publish-subscribe

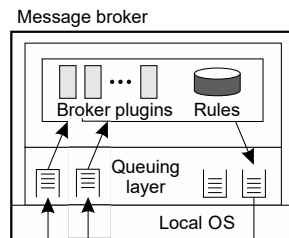
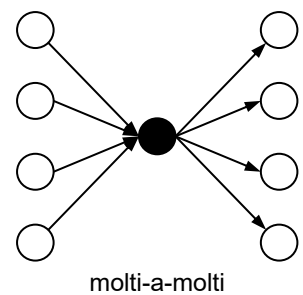
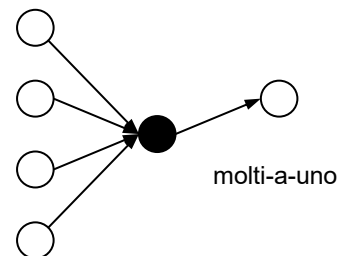
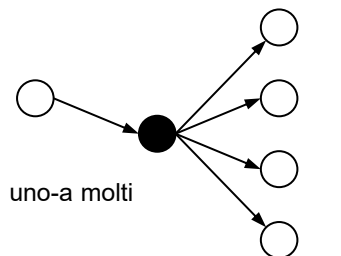
- Un message broker può essere anche adottato per effettuare un **matching** tra messaggi inviati dai mittenti e relativi destinatari
 - Adozione di un pattern **publish-subscribe**
- Disaccoppiamento tra mittente (**publisher**) e destinatario (**subscriber**)
 - Un **broker** fornisce un servizio di smistamento di messaggi relativi a un certo argomento (**topic**)
 - I **publisher** (pubblicatori) si registrano dichiarando: «voglio inviare messaggi su un certo argomento»
 - I **subscriber** (sottoscrittori) si registrano dichiarando: «voglio ricevere messaggi su un certo argomento»
- Caratteristiche:
 - Il mittente e il destinatario non comunicano direttamente (indipendenza)
 - E' possibile realizzare comunicazioni uno-a-molti, molti-a-uno, molti-a-molti
 - E' possibile realizzare comunicazioni persistenti



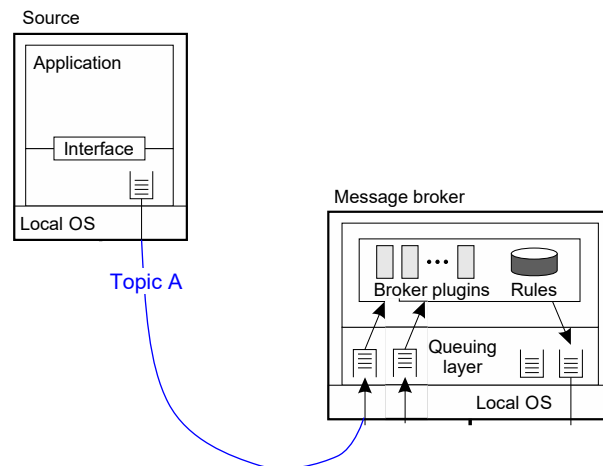
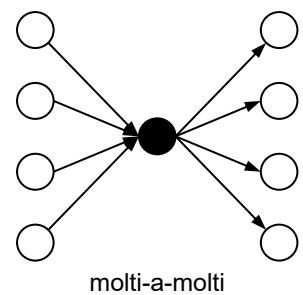
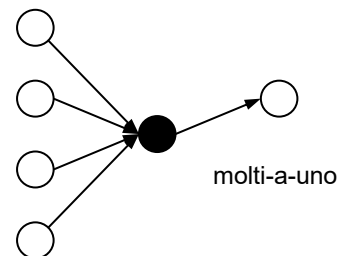
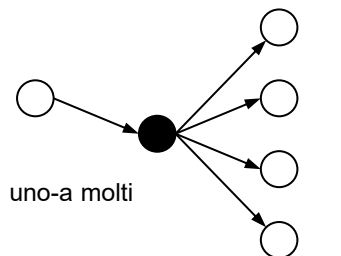
Pattern publish-subscribe



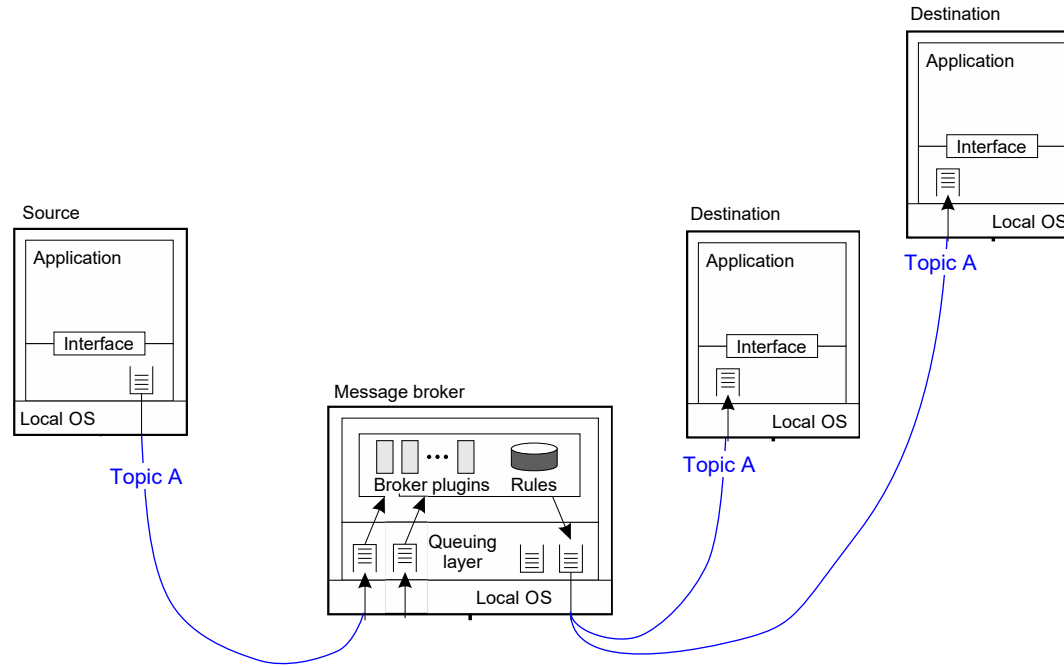
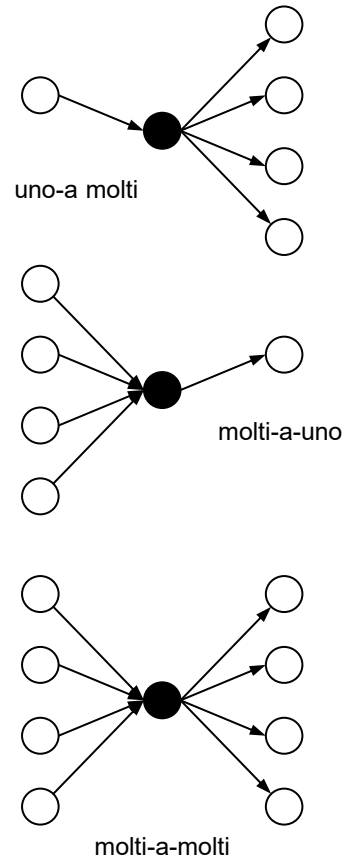
Pattern publish-subscribe



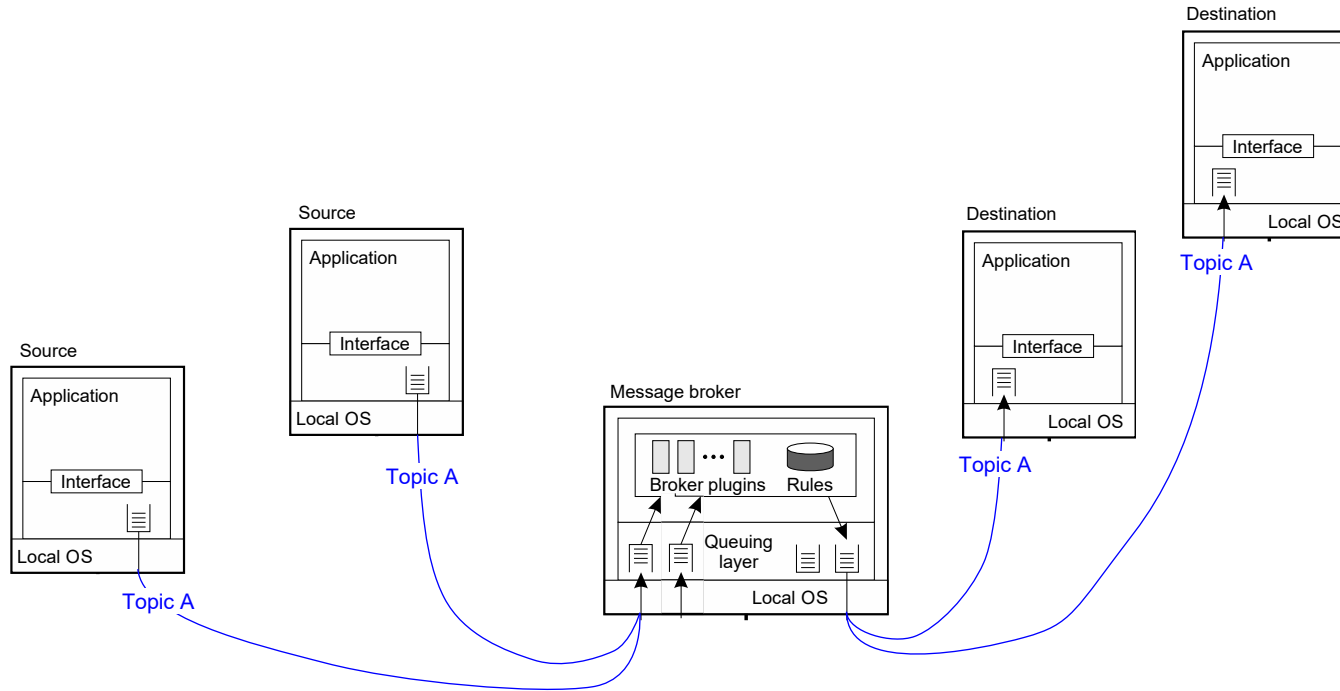
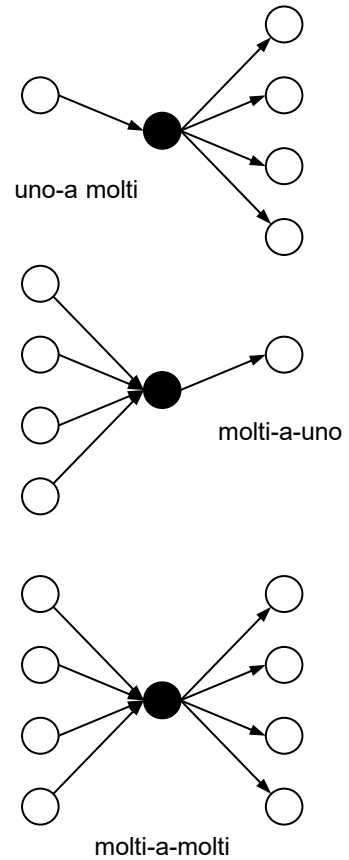
Pattern publish-subscribe



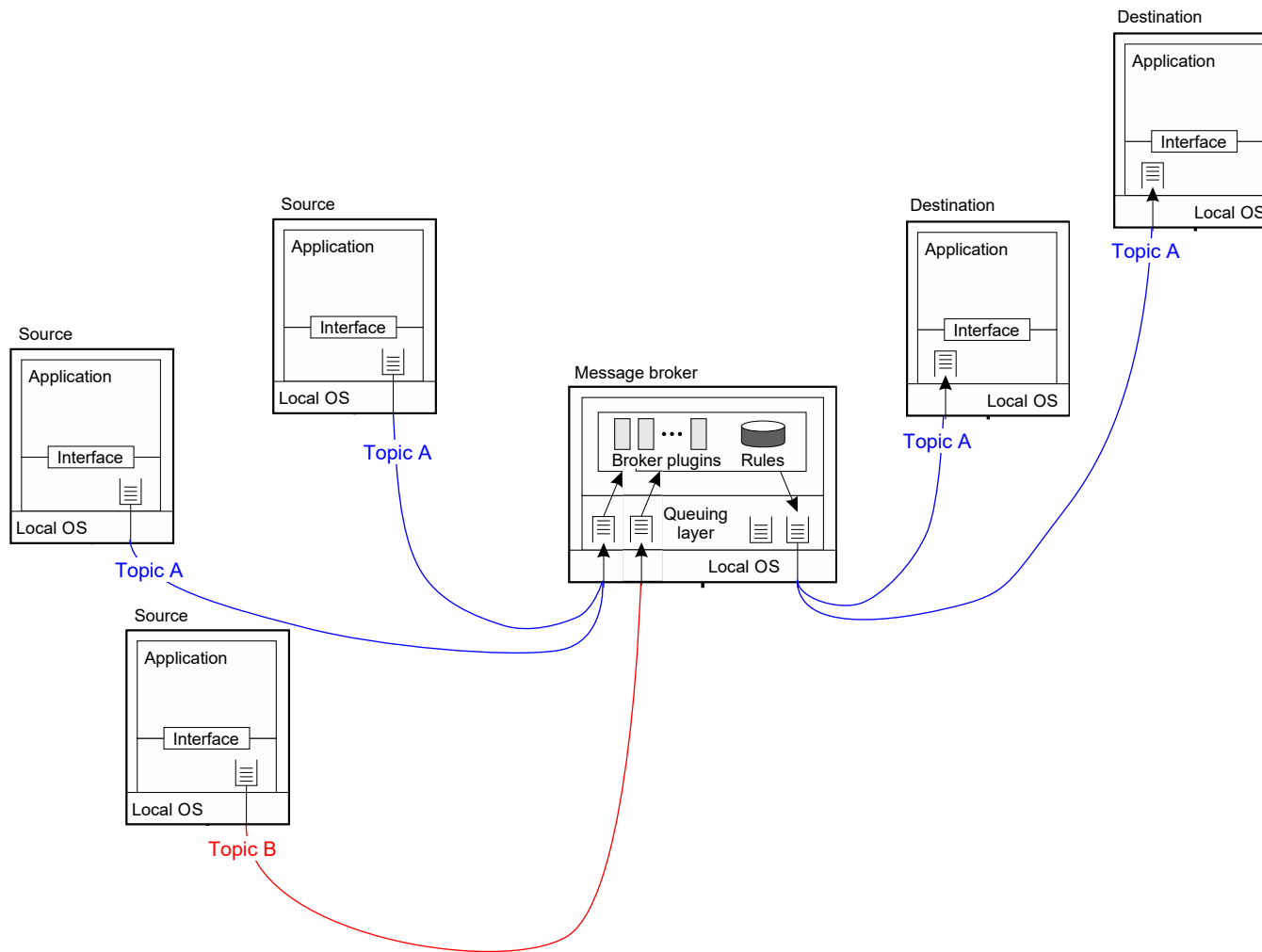
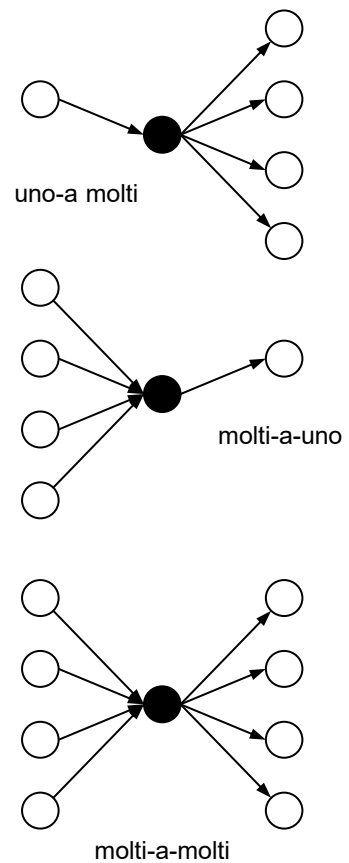
Pattern publish-subscribe



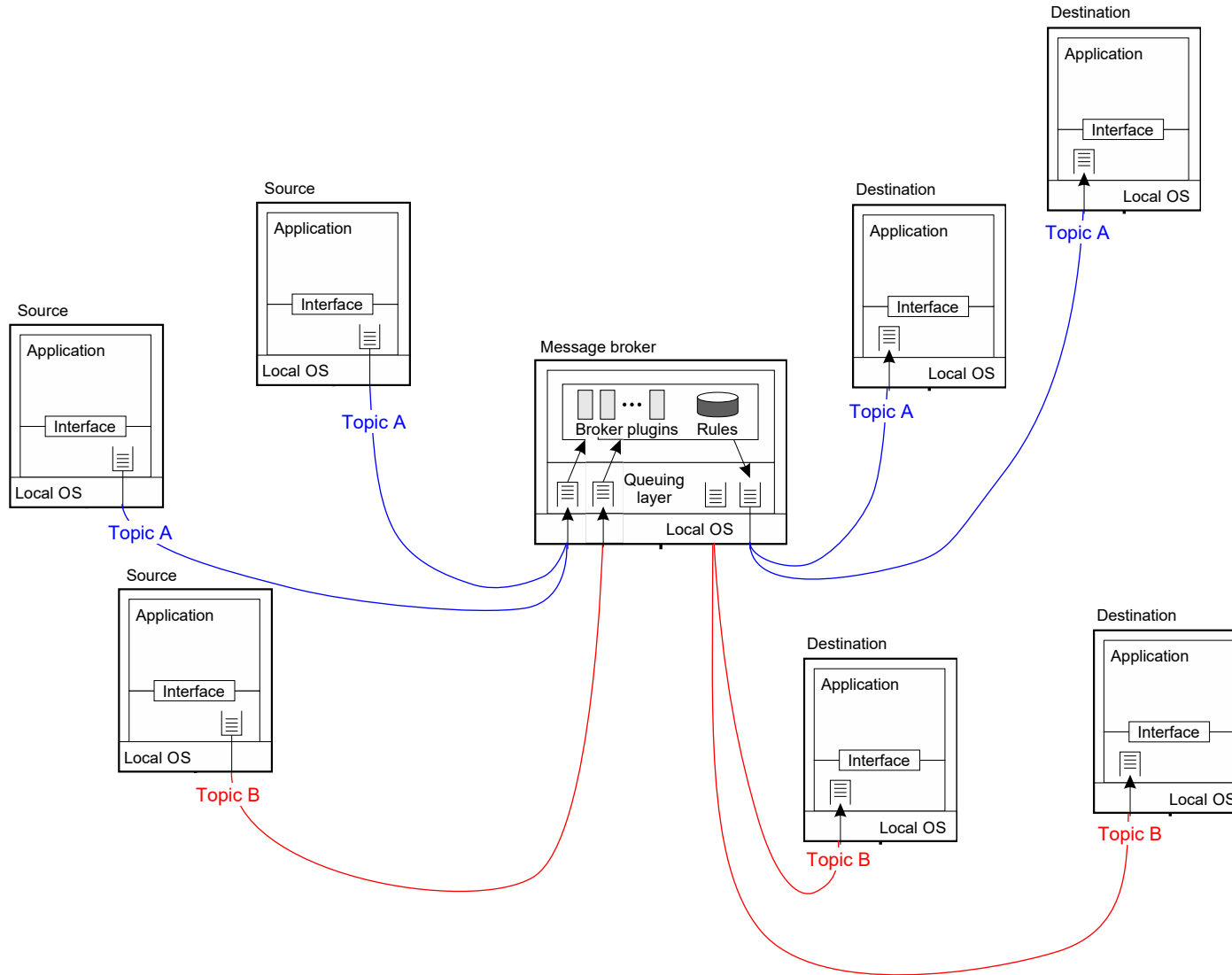
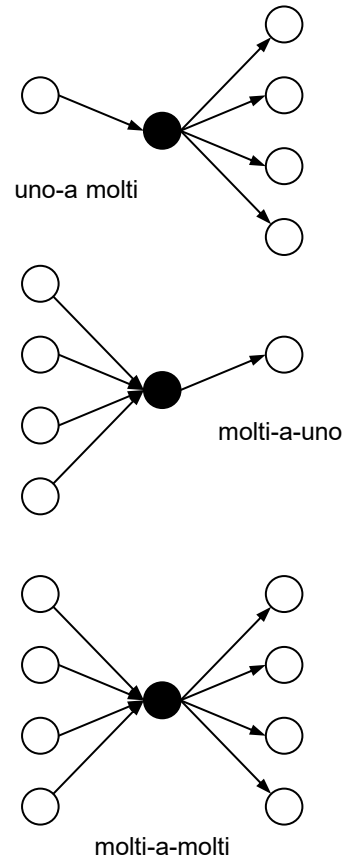
Pattern publish-subscribe



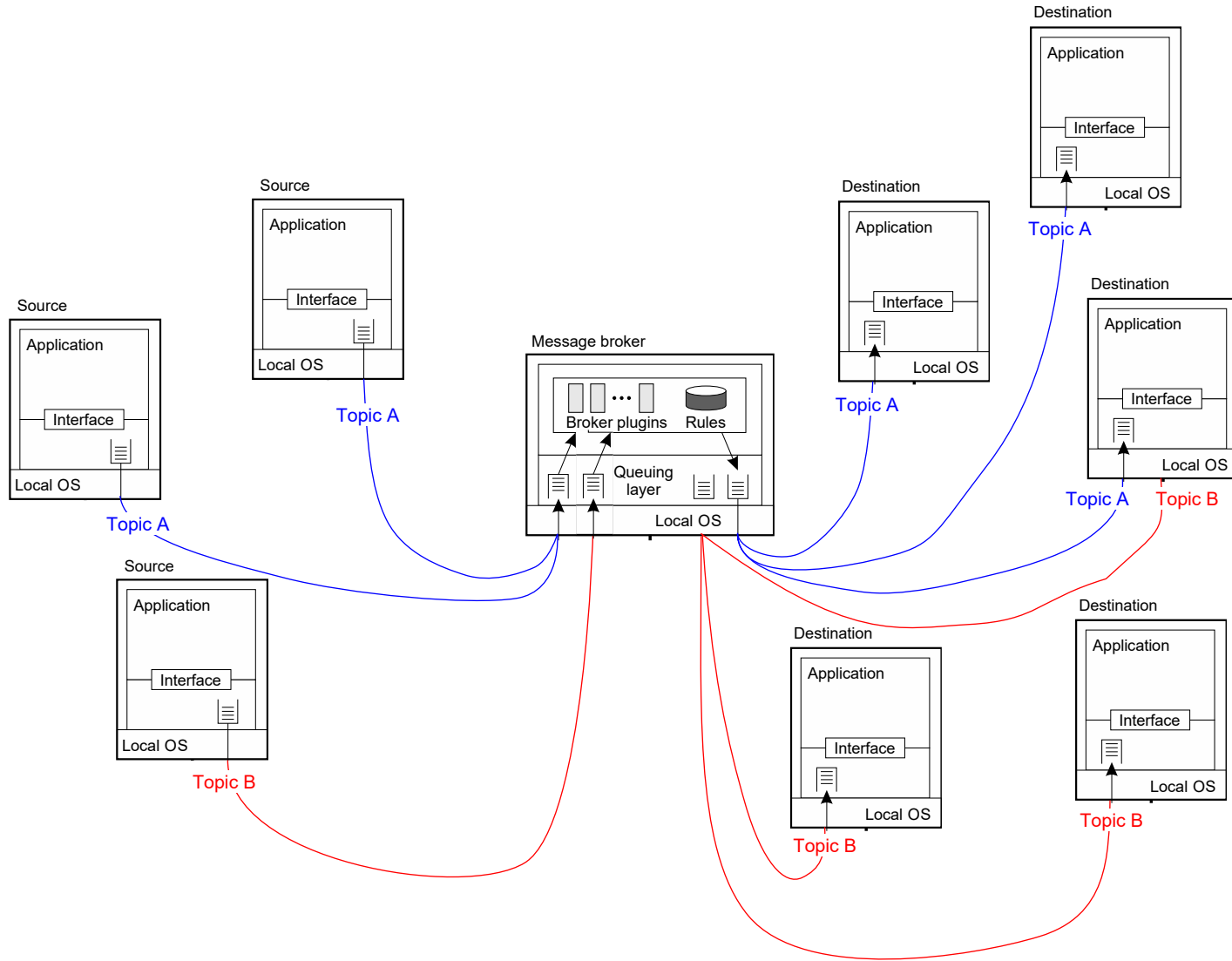
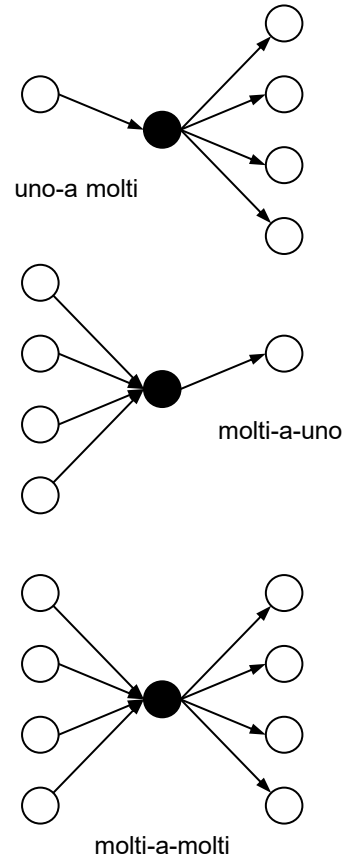
Pattern publish-subscribe



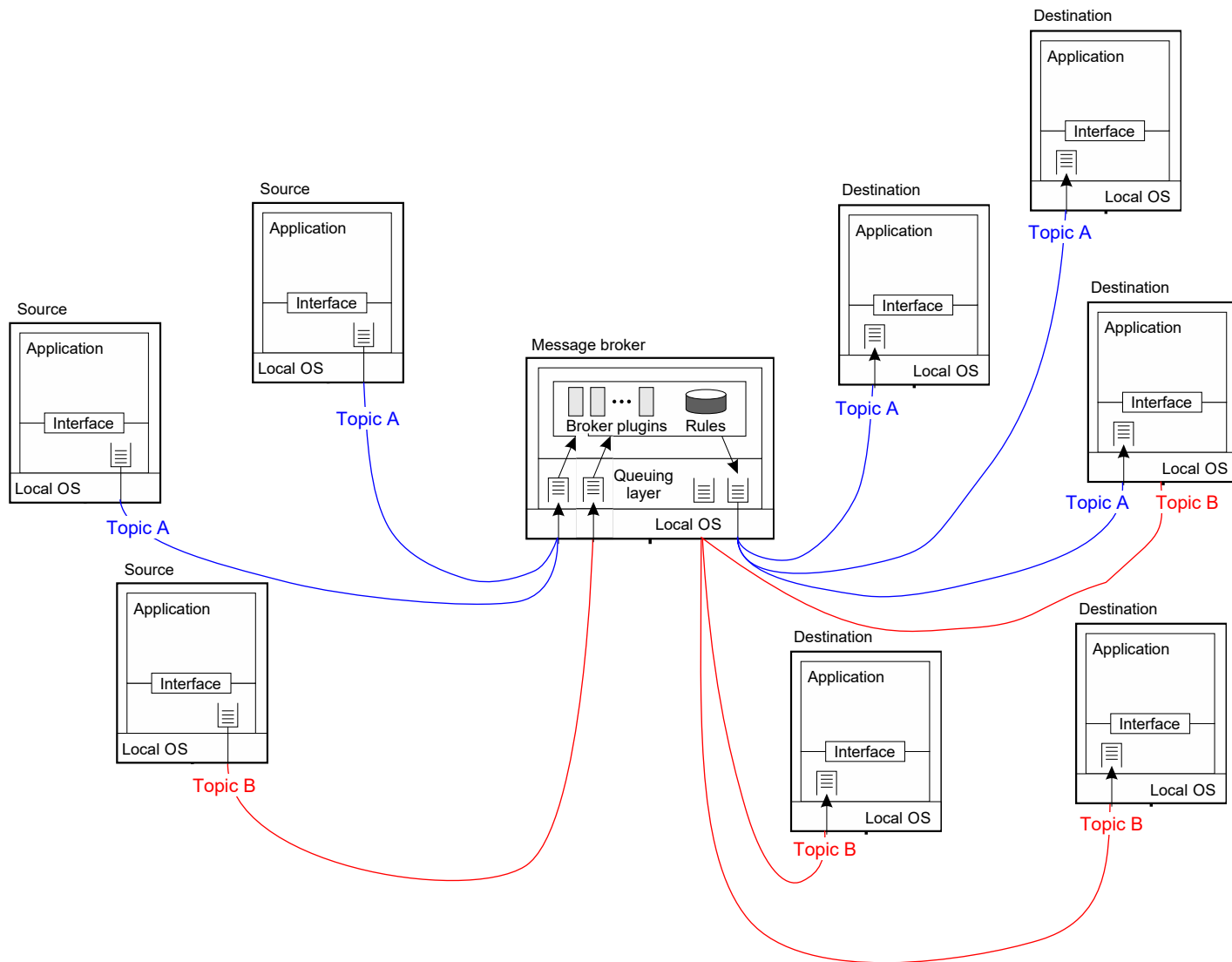
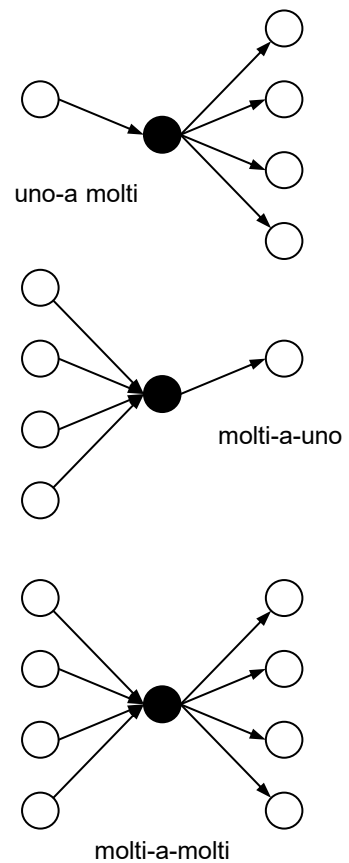
Pattern publish-subscribe



Pattern publish-subscribe

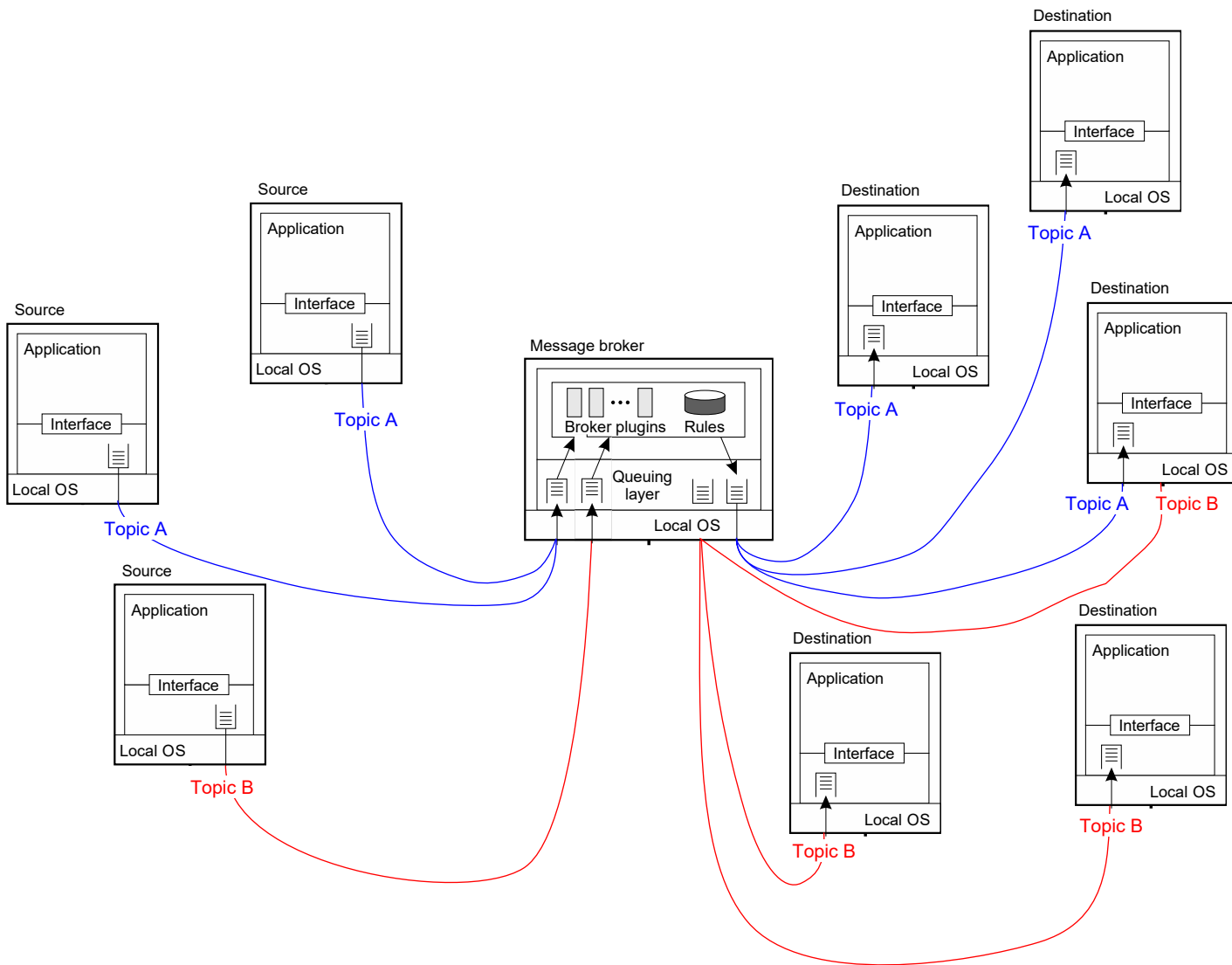
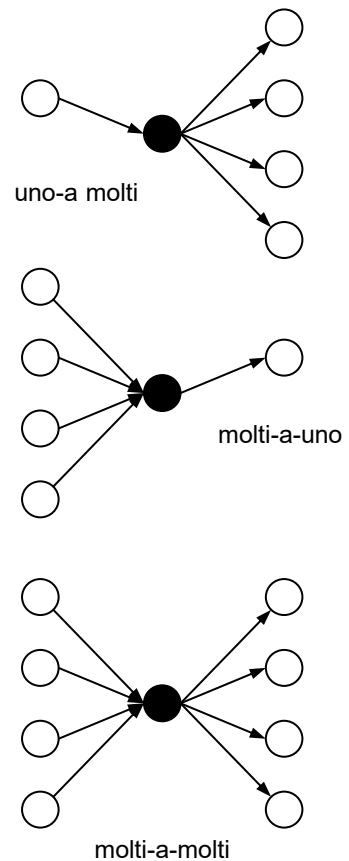


Pattern publish-subscribe



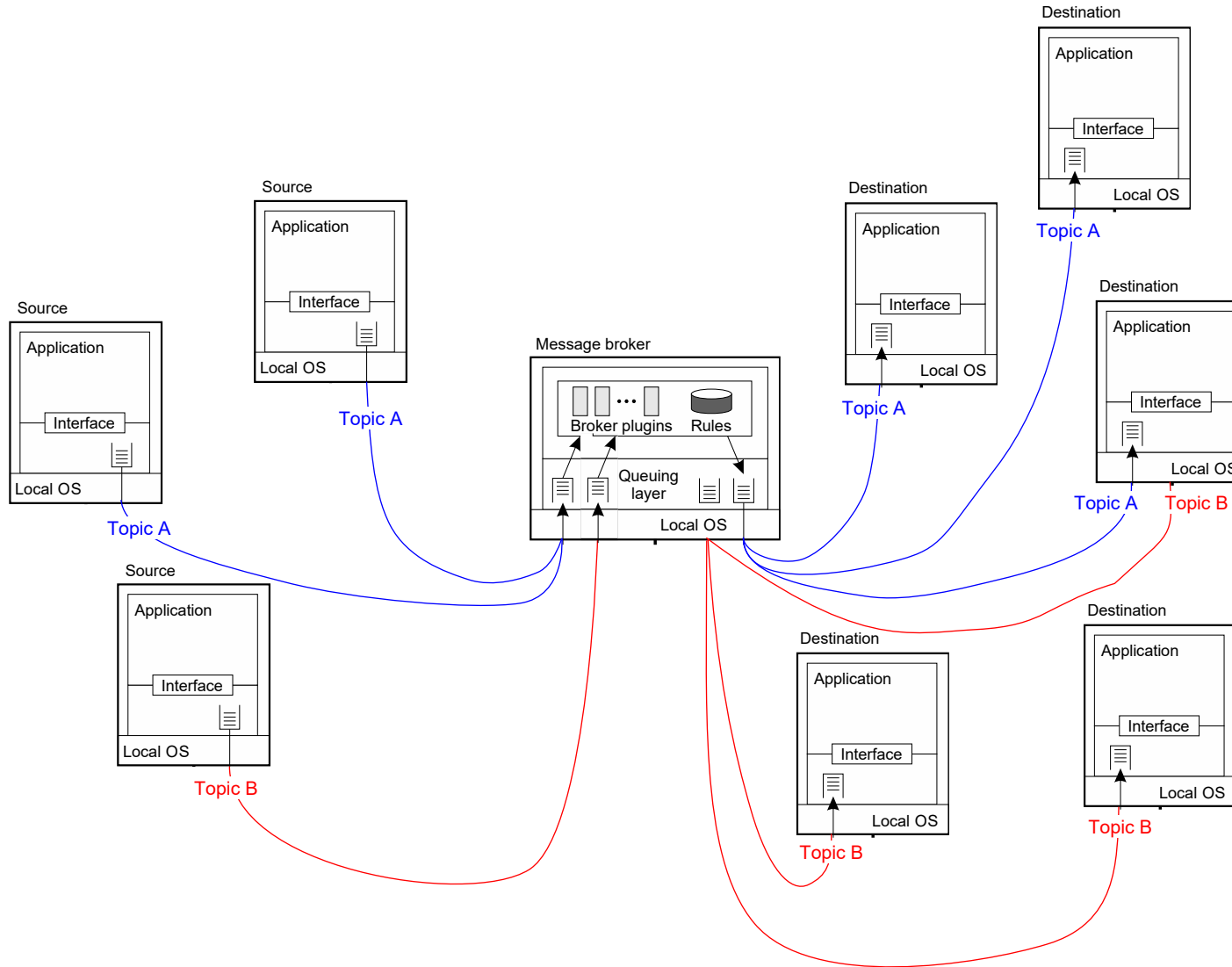
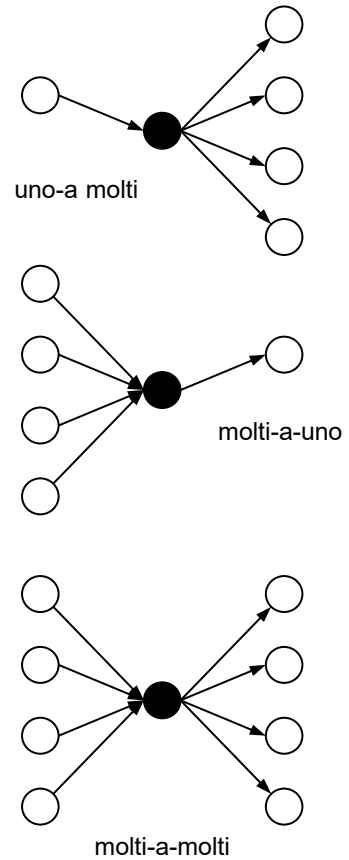
- Se nessun sottoscrittore è presente per il topic B, il broker mantiene i dati

Pattern publish-subscribe



- Se nessun sottoscrittore è presente per il topic B, il broker mantiene i dati
- Le politiche adottate possono essere diverse:

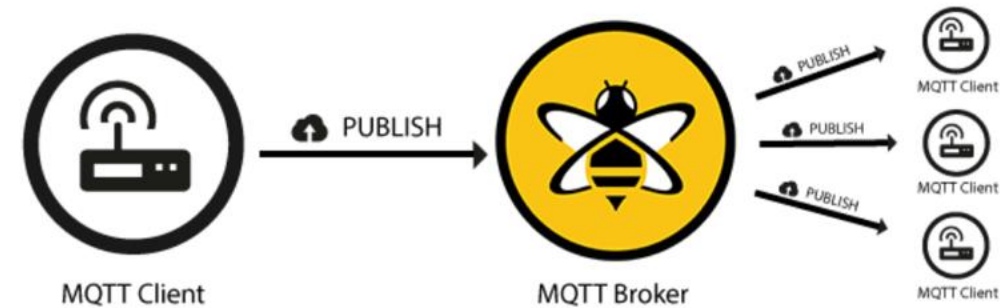
Pattern publish-subscribe



- Se nessun sottoscrittore è presente per il topic B, il broker mantiene i dati
- Le politiche adottate possono essere diverse:
 - Mantengo solo l'ultimo valore, mantengo i valori in un intervallo di tempo, ...

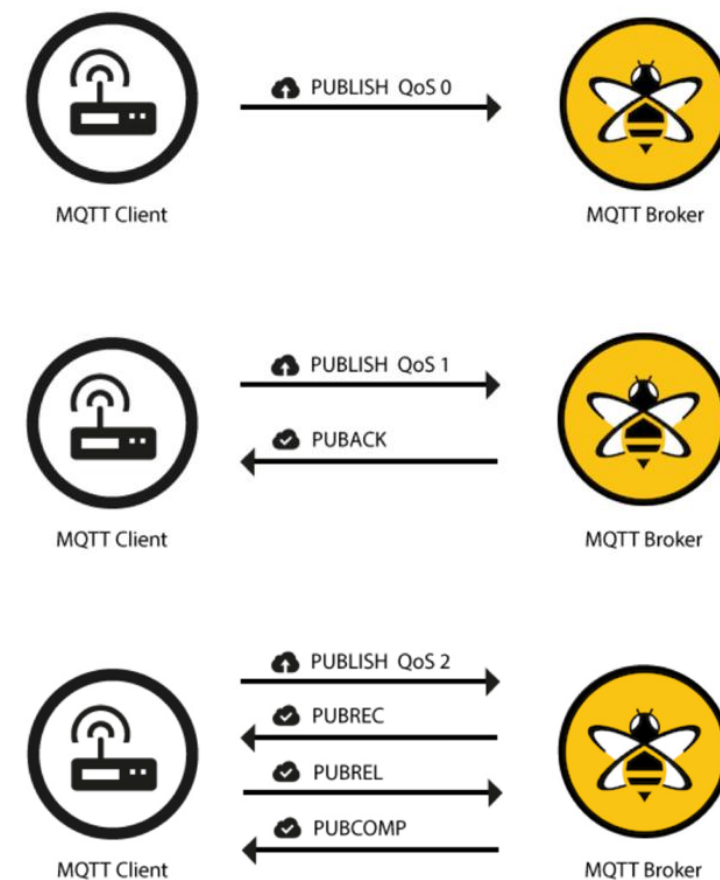
Un esempio: Message Queuing Telemetry Transport (MQTT)

- MQTT è l'implementazione più utilizzata del pattern publish-subscribe nel modo dell'**Internet of Things (IoT)**
 - Adatto sia per la comunicazione via Internet che in reti di sensori locali (es. impianto industriale)
- E' una soluzione perfetta quanto:
 - Le risorse computazionali dei dispositivi (CPU, memoria) sono limitate
 - Le risorse energetiche dei dispositivi sono limitate (alimentazione a batteria)
 - La banda è una risorsa preziosa
- Componenti:
 - **MQTT Client**: agisce da publisher e/o da subscriber
 - **MQTT Broker**: smista i messaggi (a seguito di connessione dei client)
- Link: <https://mqtt.org/>



MQTT - Quality of Service (QoS) e livelli di affidabilità

- MQTT prevede un accordo tra mittente e ricevente che definisca il **livello di affidabilità (QoS)** della comunicazione
- Tre livelli di affidabilità previsti:
 - **QoS 0: at most once (*fire and forget*)**
 - Messaggio inviato dal publisher senza conferma di ricezione
 - **QoS 1: at least once**
 - Garantisce che il messaggio arrivi (al subscriber o al broker)
 - Prevede un meccanismo di conferma (PUBACK)
 - In caso di mancata ricezione, il messaggio viene rispedito
 - Possibilità di ricezione di messaggi duplicati
 - I messaggi reinviati contengono un flag (DUP)
 - **QoS 2: exactly once**
 - Un messaggio inviato viene ricevuto una sola volta (dal subscriber o dal broker)
 - Vengono scambiati messaggi per certificare la correttezza della ricezione (*four-way handshake*)



Funzionalità di MQTT

- **Sessioni persistenti** (su richiesta da parte del subscriber)
 - Mentre il subscriber è in esecuzione: tutto funziona normalmente
 - Quando si disconnette: il broker memorizza tutti i messaggi relativi ai topic da lui sottoscritti
 - Quando si riconnette: il broker invia tutti i messaggi mantenuti in memoria
- **Messaggio “Retain”** (su richiesta da parte del publisher)
 - Il broker può mantenere uno e un solo messaggio da inviare ad ogni nuovo subscriber al momento della sua connessione (es. ultima lettura di temperatura)
 - Viene settato un flag apposito nel messaggio (`retain`)
 - Se arriva un nuovo messaggio con flag `retain` settato il vecchio viene scartato
- **Messaggio “Last Will and Testament”** (LWT, su richiesta da parte del publisher)
 - Al momento della connessione il publisher può comunicare al broker un messaggio `lwt` da inviare ai subscriber in caso di sua disconnessione inaspettata
 - Fondamentale per sistemi critici dove la mancanza di dati deve essere segnalata subito

Riferimenti bibliografici

- J. F. Kurose, K. W. Ross, Reti di Calcolatori e Internet – Un approccio top-down, 8° Edizione
 - Sezione 2.2 («Web e HTTP»), tranne sottosezione 2.2.6 («HTTP/2»)
- M. Van Steen, A. S. Tanenbaum, Distributed Systems, 4th Edition
 - Sezione 4.1.2 («Types of communication»)
 - Sezione 4.3.3 («Message-oriented persistent communication»)
- Risorsa aggiuntiva su MQTT:
 - MQTT & MQTT 5 Essentials (ebook) – Disponibile nella pagina del corso